PRD — Inventory & Order Management System (Microservices)

Oct 29, 2026 · @Abhay Purwar

1. Document metadata and version history

This PRD is version 0.1 (draft for review); it is backend-weighted at roughly 80% backend / 20% UI by requirement count.

Field	Value
Product	StockFlow — Inventory & Order Management System (working name)
Document type	Product Requirements Document, implementation-ready
Version	0.1 — Draft
Status	In review
Audience	Backend, frontend, QA, DevOps / DevSecOps engineers; tech lead; product owner
Companion artifacts (to be produced from this PRD)	Service-by-service technical design, `.proto` package, AsyncAPI event catalogue, OpenAPI spec, Postman collection, implementation backlog
Requirement ID scheme	FR, ARCH, API, GRPC, MQ, REDIS, DB, SAGA, TEST, DOCKER, SEC, SAST, OBS, NFR, UX, AC
Priority scheme	MoSCoW — **Must** (MVP, release-blocking), **Should** (MVP if time allows, else next release), **Could** (future enhancement)

Version	Date	Author	Change
0.1	2026-10-29	Abhay Purwar (with Claude)	Initial draft from master prompt

Conventions. "MUST/SHOULD/MAY" inside a requirement statement follow RFC 2119 meaning. Every **Must** requirement maps to at least one acceptance criterion in Section 19 (traceability column AC). Version numbers in Section 3 are minimum baselines; the team pins exact versions in lockfiles and image tags at Phase 1.

2. Executive summary and problem statement

StockFlow is a multi-warehouse inventory and order platform built as eight independently deployable TypeScript services that never oversell stock, survive partial failures, and prove every security and reliability claim with automated tests and Semgrep gates.

Problem. Training projects for backend engineers usually stop at CRUD over one database. Real commerce backends fail in ways CRUD never exercises: two customers buying the last unit at the same millisecond, a payment that succeeds but whose response times out, a message broker that restarts mid-publish, a refresh token stolen and replayed. Engineers who have never designed for these cases ship systems that oversell, double-charge, or leak sessions.

Solution. A platform where each of those failure modes is a first-class, tested requirement:

- **Correctness under concurrency** — inventory is reserved with atomic PostgreSQL updates guarded by `CHECK (available >= 0)`; Redis locks are never the source of truth.
- **Distributed consistency** — an orchestrated saga in the Order Service coordinates Inventory and Payment over gRPC, with idempotency keys on every mutation and compensations for every forward step.
- **Reliable events** — transactional outbox + RabbitMQ publisher confirms + idempotent consumers deliver at-least-once semantics; no exactly-once claims.
- **Secure sessions** — short-lived JWTs, rotating refresh tokens with reuse detection, Redis-backed session revocation, and durable audit in PostgreSQL with reconciliation for missed Redis expiry notifications.
- **Proven quality** — unit tests, Testcontainers-based integration tests against real PostgreSQL / Redis / RabbitMQ, contract tests for `.proto` and event schemas, E2E failure scenarios, Semgrep SAST and image scanning as merge-blocking CI gates.

Success looks like: a developer can run `make up`, place 200 concurrent orders against 50 units of stock, kill the Payment service mid-run, and observe exactly 50 confirmed orders, 150 failed or released orders, zero negative balances, and a reconciled audit trail.

3. Goals, non-goals, assumptions and constraints

The MVP proves correctness, resilience and security of the order-to-stock path; anything that does not exercise a distributed-systems or security pattern is out of scope.

3.1 Goals

ID	Goal	Measure
G-1	Zero overselling	0 negative `available` balances across the concurrency test suite (AC-010)
G-2	Every forward saga step has a compensation	100% of order failures end in `FAILED`/`CANCELLED` with stock released within 60 s or flagged by reconciliation
G-3	No lost domain events	Outbox backlog drains to 0 after a 5-minute RabbitMQ outage with no business data loss
G-4	Immediate session revocation	Revoked session rejected on next protected request (p99 < 1 s after revoke)
G-5	Security gates in CI	0 open Semgrep ERROR findings and 0 critical image CVEs on `main`
G-6	One-command local env	`make up` brings all services healthy in < 3 min on an 8 GB / 4-core laptop

3.2 Non-goals (MVP)

- Real payment gateway integration, card storage or PCI scope — payments are simulated.
- Shipping, carrier integration, tax calculation, multi-currency conversion (single currency, configured per deployment).
- Kubernetes manifests, service mesh, multi-region deployment (Could, Phase 6+).
- Customer-facing storefront; the UI is an admin/operations dashboard plus a minimal customer order view.
- Event sourcing as the system of record; events are integration events, PostgreSQL rows are the truth.
- Claims of legal or regulatory compliance (see SEC section).

3.3 Assumptions

- A-1: Single currency (default INR, configurable) stored as integer minor units (`bigint` paise/cents).
- A-2: A single order may draw from multiple warehouses; allocation picks warehouses by configured priority, then by highest available.
- A-3: Reservation TTL defaults to 15 minutes; unpaid reservations expire and are released.
- A-4: Payment is authorize-and-capture in one simulated call for MVP; split auth/capture is Should.

- A-5: Expected MVP load: 50 RPS sustained, 200 RPS burst on order creation; 10k SKUs; 20 warehouses.
- A-6: Local development runs one PostgreSQL instance with one database per service (see ARCH-004).

3.4 Constraints

- Node.js + TypeScript `strict: true`, Express 5 at the edge, gRPC internally, RabbitMQ for async, Redis, PostgreSQL.
- No cross-service database access, reads or writes.
- No secrets in images, source or committed `.env` files.
- All services containerised; CI must run the full gate set on every pull request.

3.5 Technology stack and version baselines

Baselines are minimums known to be maintained in late 2026; the team pins exact versions at Phase 1 and records them in `docs/adr/0001-versions.md`.

Category	Choice	Baseline	Rationale
Runtime	Node.js	24 LTS (22 LTS acceptable)	Active LTS; native `fetch`, stable test runner, `AbortSignal.timeout`
Language	TypeScript	5.9+, `strict`, `noUncheckedIndexedAccess`, `exactOptionalPropertyTypes`	Catch index/optional bugs at compile time
HTTP	Express	5.1+	Native async error propagation; required by brief
Validation	Zod	3.24+ / 4.x	Single schema source for REST DTOs, env config and event payloads
Data access	pg (node-postgres) + parameterized SQL, Kysely as typed query builder	pg 8.13+, Kysely 0.27+	**Decision:** explicit SQL gives full control of `SELECT … FOR UPDATE`, conditional `UPDATE … RETURNING`, `SKIP LOCKED` outbox polling and transaction boundaries; TypeORM's implicit behaviour obscures these. Kysely keeps queries typed and parameterized.
Migrations	node-pg-migrate	7.x	SQL-first, per-service migration folders

Category	Choice	Baseline	Rationale
DB	PostgreSQL	17	`MERGE`, mature `SKIP LOCKED`, JSONB
Cache / sessions	Redis (or Valkey)	7.4+ / Valkey 8	Keyspace notifications, Lua, `SET NX PX`
Redis client	ioredis	5.4+	Cluster-ready, Lua `defineCommand`, separate subscriber connection
RPC	`@grpc/grpc-js` + Buf CLI + `ts-proto`	grpc-js 1.12+, buf 1.4x	Pure-JS gRPC; Buf for lint, breaking-change detection, codegen
Messaging	RabbitMQ (quorum queues) + `amqplib`	RabbitMQ 4.1+, amqplib 0.10+	Quorum queues for durability; publisher confirms via ConfirmChannel
Auth	`jose`	5.x	Standards-compliant JWT/JWS, algorithm allow-listing
Password hashing	Argon2id (`@node-rs/argon2`)	2.x	OWASP-recommended KDF
Object storage	MinIO (S3-compatible) + `@aws-sdk/client-s3`	MinIO 2025+, SDK v3	Private buckets, presigned URLs
Logging	pino	9.x	Structured JSON, redaction paths
Observability	OpenTelemetry JS SDK, Prometheus `prom-client`, Jaeger/Tempo, Grafana	OTel 1.x / 2.x	Vendor-neutral traces + metrics
Unit tests	Vitest	3.x	Fast ESM-native TS testing (Jest 30 acceptable; pick one repo-wide)
API / integration	Supertest + Testcontainers for Node	Supertest 7, Testcontainers 10+	Real PostgreSQL / Redis / RabbitMQ / MinIO per test run

Category	Choice	Baseline	Rationale
Lint / format	ESLint (flat config, `typescript-eslint`) + Prettier	ESLint 9, Prettier 3	Includes `eslint-plugin-security`
SAST	Semgrep CE CLI	1.9x+	Required by brief; rules pinned in repo
Containers	Docker Engine + Compose v2	Docker 27+, Compose 2.29+	Healthcheck-gated `depends_on`
Image scan	Trivy	0.5x+	Image + dependency + secret scan
Secret scan	Gitleaks	8.x	Pre-commit and CI
CI	GitHub Actions (GitLab CI equivalent documented)	—	Matrix per service
API docs	OpenAPI 3.1 + Postman v2.1 collection; AsyncAPI 3.0 for events	—	Contract-first
Repo layout	**Monorepo** with pnpm workspaces + Turborepo	pnpm 9, turbo 2	Shared `proto/`, `packages/contracts`, independent Dockerfiles and versions per service (decision open: OQ-01)

4. User roles and permissions

Authorization is permission-based (`resource:action`), roles are named bundles of permissions, and every service re-checks permissions locally from JWT claims — the gateway check is a first filter, never the only one.

4.1 Roles

Role	Who	Scope
`customer`	External buyer	Own orders, own payments, own notifications, own sessions

Role	Who	Scope
`warehouse_operator`	Floor staff	Stock adjustments and transfers in assigned warehouses only
`inventory_manager`	Stock planner	All inventory, warehouses, low-stock alerts, product read
`catalog_manager`	Merchandiser	Products, categories, product media
`order_manager`	Customer-ops agent	All orders, cancellations, refunds up to a configured limit
`finance`	Finance analyst	Payments, refunds, payment reports
`auditor`	Internal audit / security	Read-only audit logs, session history, reports
`admin`	Platform administrator	User and role management, session revocation, system health
`service:<name>`	Machine identity	Internal gRPC calls only; never issued to humans

4.2 Permission matrix

`R` read, `W` create/update, `D` delete/archive, `O` own records only, `A` assigned warehouses only, — none.

Permission area	customer	wh_operator	inv_manager	catalog_mgr	order_mgr	finance	auditor	admin
`catalog:product`	R	R	R	RWD	R	R	R	RWD
`catalog:category`	R	R	R	RWD	R	—	R	RWD
`inventory:warehouse`	—	R(A)	RWD	—	R	—	R	RWD
`inventory:stock`	—	R(A)	R	R	R	—	R	R
`inventory:adjust`	—	W(A)	W	—	—	—	—	W
`inventory:transfer`	—	W(A)	W	—	—	—	—	W
`order:order`	RW(O)	—	R	—	RW	R	R	RW

Permission area	customer	wh_operator	inv_manager	catalog_mgr	order_mgr	finance	auditor	admin
`order:cancel`	W(O, pre-ship)	—	—	—	W	—	—	W
`payment:payment`	R(O)	—	—	—	R	R	R	R
`payment:refund`	—	—	—	—	W (≤ limit)	W	—	W
`notification:notification`	R(O)	R(O)	R(O)	R(O)	R(O)	R(O)	R(O)	R(O)
`audit:event`	—	—	—	—	—	—	R	R
`identity:user`	R(O)	R(O)	R(O)	R(O)	R(O)	R(O)	R	RWD
`identity:session`	RD(O)	RD(O)	RD(O)	RD(O)	RD(O)	RD(O)	R	RD
`ops:health`, `ops:metrics`	—	—	—	—	—	—	R	R

4.3 Authorization rules

- **FR-AUTHZ-01 (Must)** Permissions are embedded in the access token as a compact `perms` claim plus `wh` (assigned warehouse IDs) for scoped roles; tokens live 10 minutes so role changes take effect within one token lifetime, and an admin role change also revokes the user's sessions for immediate effect.
- **FR-AUTHZ-02 (Must)** Ownership checks (`O`) and warehouse scoping (`A`) are enforced inside the owning service's query (e.g. `WHERE customer_id = $sub`), not by filtering after fetch.
- **FR-AUTHZ-03 (Must)** Deny by default: a route or RPC without a declared permission fails closed at startup (route registry lint).
- **FR-AUTHZ-04 (Should)** `Identity.CheckPermission` RPC is used only for high-risk actions (refunds above limit, role changes) where an online check against the latest role state is required.

5. Functional requirements

The MVP comprises 52 functional requirements across eight domains: 44 Must, 5 Should, 3 Could. Each Must links to its acceptance criterion in Section 19.

5.1 Identity, sessions and access

ID	Requirement	Priority	Owner	AC
FR-001	Users register with email + password (Argon2id, min 12 chars, breached-password check Could); email unique case-insensitively. Customers self-register; staff accounts are created by `admin`.	Must	Identity	AC-001
FR-002	Login returns a 10-min access JWT and a 7-day rotating refresh token (HttpOnly, Secure, SameSite=Strict cookie for browser; body for API clients).	Must	Identity	AC-001
FR-003	Refresh rotates the token: old token is invalidated, new token issued in the same family; absolute family lifetime 30 days.	Must	Identity	AC-002
FR-004	Reuse of a rotated refresh token revokes the entire token family and its session, emits `identity.session.revoked` (reason `token_reuse`) and a security audit event.	Must	Identity	AC-003
FR-005	Users list their active sessions (device, IP prefix, created, last seen) and revoke any; admin can revoke any user's sessions.	Must	Identity	AC-004, AC-005
FR-006	Revoked or expired sessions are rejected on the next protected request across all services.	Must	Identity, Gateway	AC-004
FR-007	Session expiry produces a durable audit record within 5 minutes whether or not the Redis expiry notification was received.	Must	Identity, Audit	AC-006
FR-008	Login is throttled: 5 failures per account per 15 min triggers a progressive delay; 20 failures per IP per 15 min → 429.	Must	Identity, Gateway	AC-029
FR-009	Admin manages roles and assigned warehouses; changes are audited and force session revocation of the affected user.	Must	Identity	AC-030, AC-039
FR-010	MFA (TOTP) for staff roles.	Could	Identity	—

5.2 Catalog

ID	Requirement	Priority	Owner	AC
FR-011	CRUD products: SKU (unique, `^[A-Z0-9-]{3,40}$`), name, description, category, unit price (minor units > 0), status `DRAFT/ACTIVE/ARCHIVED`, low-stock threshold default.	Must	Catalog	AC-007
FR-012	CRUD hierarchical categories (max depth 4); a category with products cannot be deleted, only archived.	Must	Catalog	AC-007
FR-013	Search products by text (name/SKU), category, status, price range; sort by name, price, created; cursor pagination, max page size 100.	Must	Catalog	AC-009, AC-045
FR-014	Product reads are cache-aside with invalidation on write; stale reads bounded by TTL.	Must	Catalog	AC-008
FR-015	Upload product images/spec documents to a private bucket via presigned PUT; allow-listed MIME types (jpeg, png, webp, pdf), max 10 MB, content sniffing on finalize; reads via short-lived presigned GET.	Should	Catalog	AC-040
FR-016	Price changes never alter existing orders (orders hold a price snapshot).	Must	Catalog, Order	AC-046

5.3 Inventory and warehouses

ID	Requirement	Priority	Owner	AC
FR-017	CRUD warehouses (code, name, address, priority, status `ACTIVE/INACTIVE`).	Must	Inventory	AC-012
FR-018	Track per-warehouse balance per SKU: `on_hand`, `reserved`, `available = on_hand − reserved` (generated column); `available` never negative.	Must	Inventory	AC-010
FR-019	Reserve stock for an order atomically across one or more warehouses; all-or-nothing per order.	Must	Inventory	AC-010
FR-020	Confirm (commit) a reservation: decrement `on_hand` and `reserved` together; write movement `SALE`.	Must	Inventory	AC-016
FR-021	Release a reservation (payment failed, cancelled, expired): decrement `reserved`; idempotent.	Must	Inventory	AC-011, AC-017

ID	Requirement	Priority	Owner	AC
FR-022	Reservations expire after TTL (default 15 min) and are released by a sweeper; Order Service is notified via `inventory.stock.released` (reason `expired`).	Must	Inventory	AC-011
FR-023	Manual adjustments with mandatory reason code (`RECEIPT`, `DAMAGE`, `COUNT_CORRECTION`, `RETURN`) and note; cannot drive `available` below 0.	Must	Inventory	AC-012
FR-024	Transfers between warehouses are a single transaction producing paired `TRANSFER_OUT` / `TRANSFER_IN` movements.	Must	Inventory	AC-013
FR-025	Immutable movement ledger: every balance change has one movement row with actor, reason, correlation ID; `SUM(movements) = on_hand` is verifiable.	Must	Inventory	AC-012
FR-026	Low-stock detection when `available` crosses below threshold (edge-triggered, re-armed when it rises above threshold × 1.2).	Must	Inventory	AC-014
FR-027	Two-step transfers (in-transit state).	Could	Inventory	—

5.4 Orders

ID	Requirement	Priority	Owner	AC
FR-028	Create order with 1–50 lines (qty 1–1000 each); requires `Idempotency-Key` header (UUID); same key + same payload returns the original order, different payload returns 409.	Must	Order	AC-015
FR-029	Each line stores SKU, product name and unit price snapshot from Catalog at creation time; total computed server-side.	Must	Order	AC-046
FR-030	Order follows the state machine in Section 11; invalid transitions are rejected with 409 `INVALID_STATE_TRANSITION`.	Must	Order	AC-020
FR-031	Order saga reserves stock, processes payment, confirms stock, confirms order; failure at any step triggers compensation.	Must	Order	AC-016, AC-017
FR-032	Customer can cancel own order while `PENDING`, `RESERVED` or `CONFIRMED` (before fulfilment);	Must	Order	AC-019

ID	Requirement	Priority	Owner	AC
	`order_manager` can cancel until `FULFILLED`. Paid orders are refunded.			
FR-033	Order history: every state change is stored with timestamp, actor, reason.	Must	Order	AC-020
FR-034	List/filter orders by status, customer, date range, total range; cursor pagination.	Must	Order	AC-045
FR-035	Mark order `FULFILLED` (manual, ops).	Should	Order	—

5.5 Payments (simulated)

ID	Requirement	Priority	Owner	AC
FR-036	`ProcessPayment` simulates authorize+capture; outcome controlled by test tokens (`pm_ok`, `pm_declined`, `pm_timeout`, `pm_slow_success`, `pm_error`) and a configurable random failure rate.	Must	Payment	AC-016, AC-017, AC-018
FR-037	Payment requests are idempotent by `idempotency_key` (= order ID + attempt); repeated calls return the original result.	Must	Payment	AC-018
FR-038	Full and partial refunds; total refunded ≤ captured amount (DB constraint); idempotent by refund key.	Must	Payment	AC-021
FR-039	Payment status query by order ID for reconciliation.	Must	Payment	AC-018
FR-040	No PAN, CVV or real credentials accepted or stored; request schema rejects card-number-like strings.	Must	Payment	AC-042

5.6 Notifications

ID	Requirement	Priority	Owner	AC
FR-041	Create in-app notifications from events: order confirmed/failed/cancelled, payment failed/refunded (customer); low stock (inventory managers); session revoked for reuse (user + admin).	Must	Notification	AC-022
FR-042	List own notifications, unread count, mark read / mark all read.	Must	Notification	AC-022
FR-043	Exactly one notification per (event ID, recipient) despite duplicate delivery.	Must	Notification	AC-022

ID	Requirement	Priority	Owner	AC
FR-044	Email/SMS channel adapters (stubbed).	Could	Notification	—

5.7 Audit and reporting

ID	Requirement	Priority	Owner	AC
FR-045	Record audit events for: login success/failure, logout, refresh reuse, session revoke/expire, role change, stock adjustment, transfer, order state change, refund.	Must	Audit	AC-039
FR-046	Audit events are append-only (DB role has no UPDATE/DELETE), searchable by actor, type, resource, date.	Must	Audit	AC-039
FR-047	Daily summaries (orders by status, revenue, refunds, sessions created/expired/revoked) computed from consumed events, idempotent on rerun.	Should	Audit	AC-044
FR-048	Ops metrics view: queue depth, DLQ count, outbox backlog, cache hit rate (proxied from Prometheus).	Should	Gateway, Audit	AC-037

5.8 Platform

ID	Requirement	Priority	Owner	AC
FR-049	Every request carries a correlation ID end-to-end (HTTP → gRPC metadata → AMQP headers → logs, traces).	Must	All	AC-038
FR-050	Health (`/livez`, `/readyz`) on every service.	Must	All	AC-037
FR-051	DLQ inspection and replay via admin CLI.	Should	Ops tooling	AC-023
FR-052	Reconciliation jobs for stuck sagas, expired reservations, missed session expiries; results exposed as metrics.	Must	Order, Inventory, Identity	AC-018, AC-006

6. Microservices architecture and ownership

Eight services each own one bounded context and one database; the Order Service orchestrates the only cross-service write path, and the synchronous call graph is acyclic.

6.1 Service catalogue

Service	Owns (data)	Exposes	Calls (gRPC)	Publishes	Consumes
api-gateway (BFF)	Nothing persistent (rate-limit counters in Redis `gw:` namespace)	REST `/api/v1/*`	All services' public RPCs; `Identity.ValidateSession`	—	—
identity-svc	users, roles, permissions, user_roles, refresh token families, session index (Redis), auth audit outbox	gRPC `identity.v1`	—	`identity.session.*`, `identity.user.*`, `audit.security.*`	—
catalog-svc	products, categories, product_media	gRPC `catalog.v1`	—	`catalog.product.*`	—
inventory-svc	warehouses, inventory_balances, reservations, movements, low_stock_state	gRPC `inventory.v1`	`Catalog.ValidateProduct`	`inventory.stock.*`	`catalog.product.archived`
order-svc	orders, order_items, order_status_history, saga_state	gRPC `order.v1`	`Catalog.GetProducts`, `Inventory.*`, `Payment.*`	`order.*`	`inventory.stock.released`, `payment.*` (reconciliation)
payment-svc	payments, payment_attempts, refunds	gRPC `payment.v1`	—	`payment.*`	—

Service	Owns (data)	Exposes	Calls (gRPC)	Publishes	Consumes
notification-svc	notifications, processed_events	gRPC `notification.v1`	—	—	`order.*`, `payment.*`, `inventory.stock.low`, `identity.session.revoked`
audit-svc (Audit & Reporting)	audit_events (append-only), daily_summaries, processed_events	gRPC `audit.v1`	—	—	# on `domain.events` + `audit.*`

Each service with an outbox runs a separate **outbox-relay** process from the same image (`node dist/outbox-relay.js`), and Inventory, Order and Identity run a **reconciler** process. Workers scale independently of the gRPC server.

6.2 Architecture requirements

ID	Requirement	Priority
ARCH-001	Each service is an independent package with its own `package.json`, Dockerfile, migrations, config schema and semantic version.	Must
ARCH-002	A service reads and writes only its own database; another service's data is obtained via gRPC or replicated from events.	Must
ARCH-003	Each service connects with its own PostgreSQL role, granted privileges only on its own database.	Must
ARCH-004	Local dev uses one PostgreSQL instance with one database per service (`identity_db`, `catalog_db`, …); production may use separate instances without code changes (connection string per service).	Must
ARCH-005	Internal synchronous communication is gRPC only; no service-to-service REST. The gateway is the only HTTP surface for clients.	Must
ARCH-006	The synchronous call graph is a DAG: Gateway → {Order, Catalog, Inventory, Payment, Identity, Notification, Audit}; Order → {Catalog, Inventory, Payment}; Inventory → Catalog. Inventory and Payment never call Order; they publish events. Enforced by a dependency lint on generated clients.	Must

ID	Requirement	Priority
ARCH-007	Asynchronous domain events go through RabbitMQ with versioned schemas in `packages/contracts/events`.	Must
ARCH-008	The gateway contains no business rules (no stock, price or state logic); it maps, authenticates, authorizes coarsely and rate-limits.	Must
ARCH-009	Services are stateless; all state lives in PostgreSQL, Redis or RabbitMQ, so any replica can serve any request.	Must
ARCH-010	Shared code is limited to `packages/` libraries (logger, config, grpc-kit, mq-kit, outbox, errors, contracts) with no domain logic.	Must
ARCH-011	Architecture Decision Records (ADRs) are kept in `docs/adr/` for every choice in Sections 3.5 and 6.	Should

6.3 High-level architecture(you can view this in mermaid.io)

```
flowchart LR
  subgraph Clients
    UI[Admin Dashboard SPA]
    API[API clients / Postman]
  end
  UI -->|HTTPS REST| GW
  API -->|HTTPS REST| GW
  GW[API Gateway / BFF<br/>Express 5]
  GW -->|gRPC mTLS| ID[Identity]
  GW -->|gRPC| CAT[Catalog]
  GW -->|gRPC| INV[Inventory]
  GW -->|gRPC| ORD[Order<br/>saga orchestrator]
  GW -->|gRPC| PAY[Payment Sim]
  GW -->|gRPC| NOT[Notification]
  GW -->|gRPC| AUD[Audit & Reporting]
  ORD -->|gRPC| CAT
  ORD -->|gRPC| INV
  ORD -->|gRPC| PAY
  INV -->|gRPC| CAT
  subgraph Data
    PG[(PostgreSQL<br/>db per service)]
    RD[(Redis)]
    S3[(MinIO private bucket)]
  end
  ID --- PG
  CAT --- PG
  INV --- PG
  ORD --- PG
  PAY --- PG
  NOT --- PG
  AUD --- PG
  ID --- RD
  CAT --- RD
  GW --- RD
```

```
  CAT --- S3
  MQ{{RabbitMQ<br/>domain.events topic}}
  ID -. outbox .-> MQ
  CAT -. outbox .-> MQ
  INV -. outbox .-> MQ
  ORD -. outbox .-> MQ
  PAY -. outbox .-> MQ
  MQ -.-> NOT
  MQ -.-> AUD
  MQ -.-> ORD
  MQ -.-> INV
```

Solid arrows are synchronous gRPC with deadlines; dotted arrows are asynchronous, at-least-once events.

6.4 Service ownership and data boundaries

```
flowchart TB
  subgraph identity_db
    users; roles; permissions; refresh_token_families; id_outbox[outbox]
  end
  subgraph catalog_db
    products; categories; product_media; cat_outbox[outbox]
  end
  subgraph inventory_db
    warehouses; inventory_balances; reservations; reservation_lines; movements;
inv_outbox[outbox]; inv_processed[processed_events]
  end
  subgraph order_db
    orders; order_items; order_status_history; saga_state; ord_outbox[outbox];
ord_processed[processed_events]
  end
  subgraph payment_db
    payments; payment_attempts; refunds; pay_outbox[outbox]
  end
  subgraph notification_db
    notifications; not_processed[processed_events]
  end
  subgraph audit_db
    audit_events; daily_summaries; aud_processed[processed_events]
  end
  IDS[identity-svc] --> identity_db
  CATS[catalog-svc] --> catalog_db
  INVS[inventory-svc] --> inventory_db
  ORDS[order-svc] --> order_db
  PAYS[payment-svc] --> payment_db
  NOTS[notification-svc] --> notification_db
  AUDS[audit-svc] --> audit_db
```

Cross-service references are stored as opaque IDs without foreign keys (e.g. `order_items.product_id`, `reservations.order_id`). They are validated at write time by gRPC (`Catalog.GetProducts` during order creation, `Catalog.ValidateProduct` when Inventory creates a balance) and kept consistent afterwards by events (`catalog.product.archived` blocks new reservations for that SKU).

6.5 Repository layout

```
stockflow/
  proto/                      # *.proto, buf.yaml, buf.gen.yaml
  packages/
    contracts/                # generated gRPC TS + Zod event schemas + OpenAPI
    platform/                 # logger, config, errors, otel, health
    grpc-kit/  mq-kit/  outbox/  idempotency/  testing/
  services/
    api-gateway/ identity/ catalog/ inventory/ order/ payment/ notification/ audit/
      src/{domain,application,infrastructure,interfaces}/
      migrations/  test/{unit,integration}/  Dockerfile
  e2e/                        # cross-service scenarios
  deploy/compose/             # docker-compose*.yml, rabbitmq definitions.json
  .semgrep/                   # custom rules + tests
  docs/adr/  docs/runbooks/
```

Each service follows a ports-and-adapters layout: `domain` (pure logic, 100% unit-testable), `application` (use cases, transactions), `infrastructure` (pg, redis, amqp, grpc clients), `interfaces` (gRPC handlers, consumers).

7. REST and gRPC contracts

Clients see one versioned REST API at the gateway; behind it, typed Protocol Buffer contracts with mandatory deadlines, idempotency keys on every mutation, and an explicit rule that a timeout means "unknown", never "failed".

7.1 REST standards

ID	Requirement	Priority
API-001	Base path `/api/v1`; breaking changes go to `/api/v2` with 6-month overlap.	Must
API-002	OpenAPI 3.1 spec generated from Zod schemas (`zod-to-openapi`), published at `/api/v1/openapi.json` (non-prod) and committed; CI fails if the committed spec differs from generated.	Must
API-003	Postman collection + environment template (`baseUrl`, `accessToken`, no secrets) generated from OpenAPI and committed.	Must
API-004	Uniform error body (below) for every 4xx/5xx; never stack traces, SQL, internal hostnames or gRPC details.	Must
API-005	`X-Correlation-Id` accepted (UUID, else regenerated) and always returned.	Must
API-006	Cursor pagination: `?limit=` (default 20, max 100) `&cursor=` (opaque, base64url of last sort key + id); response `{ data, page: { nextCursor, limit } }`. Offset pagination is not offered.	Must

ID	Requirement	Priority
API-007	Filtering via explicit allow-listed query params; sorting via `sort=field` / `sort=-field` on an allow-list; unknown params → 400.	Must
API-008	`Idempotency-Key` header required on `POST /orders`, `POST /orders/{id}/cancel`, `POST /payments/{id}/refunds`, `POST /inventory/adjustments`, `POST /inventory/transfers`. Stored 24 h.	Must
API-009	Body size limit 100 KB (JSON); `Content-Type` must be `application/json` for bodies.	Must
API-010	Rate limits (Redis sliding window): 100 req/min per user, 300 req/min per IP, auth endpoints 10 req/min per IP; headers `RateLimit-Limit/Remaining/Reset`; 429 with `Retry-After`.	Must
API-011	Optimistic concurrency on updatable resources: `ETag` (row `version`) returned, `If-Match` required on `PATCH`; mismatch → 412.	Should

Error schema (RFC 9457 Problem Details shape):

```
{
  "type": "https://errors.stockflow.dev/insufficient-stock",
  "title": "Insufficient stock",
  "status": 409,
  "code": "INSUFFICIENT_STOCK",
  "detail": "Requested 5 of SKU ABC-123; 2 available.",
  "correlationId": "7d0c2b1e-4c0f-4b8e-9a51-2f0c8a4b9e11",
  "errors": [{ "path": "items[0].quantity", "message": "must be ≤ 2" }]
}
```

7.2 Endpoint catalogue

Method & path	Purpose	Permission	Idem.	gRPC	Events
`POST /auth/register`	Customer sign-up	public	—	`Identity.Register`	`identity.user.registered`
`POST /auth/login`	Issue tokens + session	public	—	`Identity.Login`	`audit.security.login_*`
`POST /auth/refresh`	Rotate refresh token	refresh cookie	—	`Identity.Refresh`	`identity.session.revoked` on reuse
`POST /auth/logout`	Revoke current session	authenticated	—	`Identity.RevokeSession`	`identity.session.revoked`
`GET /me/sessions`	List own sessions	`identity:session` R(O)	—	`Identity.ListSessions`	—

Method & path	Purpose	Permission	Idem.	gRPC	Events
`DELETE /me/sessions/{sessionId}`	Revoke own session	`identity:session` D(O)	—	`Identity.RevokeSession`	`identity.session.revoked`
`GET /admin/users` · `POST` · `PATCH /{id}`	Manage staff, roles, warehouses	`identity:user` W	—	`Identity.*User`	`identity.user.role_changed`
`DELETE /admin/users/{id}/sessions`	Revoke all sessions of a user	`identity:session` D	—	`Identity.RevokeUserSessions`	`identity.session.revoked`
`GET /products`	Search/list	`catalog:product` R	—	`Catalog.SearchProducts`	—
`GET /products/{id}`	Detail (cached)	`catalog:product` R	—	`Catalog.GetProduct`	—
`POST /products` · `PATCH /products/{id}`	Create/update	`catalog:product` W	—	`Catalog.CreateProduct/UpdateProduct`	`catalog.product.created/updated`
`POST /products/{id}/archive`	Archive	`catalog:product` D	—	`Catalog.ArchiveProduct`	`catalog.product.archived`
`POST /products/{id}/media/upload-url` · `POST /products/{id}/media/{mediaId}/finalize`	Presigned upload + verify	`catalog:product` W	—	`Catalog.*Media`	—
`GET/POST/PATCH /categories`	Category tree	`catalog:category`	—	`Catalog.*Category`	—
`GET/POST/PATCH /warehouses`	Warehouses	`inventory:warehouse`	—	`Inventory.*Warehouse`	—
`GET /inventory?sku=&warehouseId=`	Balances	`inventory:stock` R	—	`Inventory.GetInventory`	—
`GET /inventory/movements`	Ledger	`inventory:stock` R	—	`Inventory.ListMovements`	—

Method & path	Purpose	Permission	Idem.	gRPC	Events
`POST /inventory/adjustments`	Manual adjustment	`inventory:adjust`	Yes	`Inventory.AdjustStock`	`inventory.stock.adjusted`, maybe `inventory.stock.low`
`POST /inventory/transfers`	Transfer	`inventory:transfer`	Yes	`Inventory.TransferStock`	`inventory.stock.transferred`
`GET /inventory/alerts/low-stock`	Active low-stock list	`inventory:stock` R	—	`Inventory.ListLowStock`	—
`POST /orders`	Create order + start saga	`order:order` W	Yes	`Order.CreateOrder`	`order.created` → …
`GET /orders` · `GET /orders/{id}`	List / detail + history	`order:order` R	—	`Order.ListOrders/GetOrder`	—
`GET /orders/{id}/status`	Lightweight polling	`order:order` R	—	`Order.GetOrderStatus`	—
`POST /orders/{id}/cancel`	Cancel (+refund)	`order:cancel`	Yes	`Order.CancelOrder`	`order.cancelled`, `payment.refunded`
`POST /orders/{id}/fulfil`	Mark fulfilled	`order:order` W (staff)	Yes	`Order.FulfilOrder`	`order.fulfilled`
`GET /payments?orderId=` · `GET /payments/{id}`	Payment status/history	`payment:payment` R	—	`Payment.GetPaymentStatus`	—
`POST /payments/{id}/refunds`	Manual refund	`payment:refund`	Yes	`Payment.ProcessRefund`	`payment.refunded`
`GET /notifications` · `GET /notifications/unread-count`	Inbox	`notification` R(O)	—	`Notification.List/UnreadCount`	—
`POST /notifications/{id}/read` · `POST`	Mark read	`notification` R(O)	—	`Notification.MarkRead`	—

Method & path	Purpose	Permission	Idem.	gRPC	Events
`/notifications/read-all`					
`GET /audit/events`	Search audit	`audit:event R`	—	`Audit.SearchEvents`	—
`GET /reports/daily?date=` · `GET /reports/sessions`	Summaries	`audit:event R`	—	`Audit.GetDailySummary`	—
`GET /ops/health` · `GET /ops/queues`	Aggregated health, queue/DLQ/outbox stats	`ops:health`	—	each `grpc.health.v1` + RabbitMQ mgmt API	—
`GET /livez` · `GET /readyz`	Gateway probes	public (internal network)	—	—	—

Key request/response schemas (Zod is the source of truth; OpenAPI generated):

```
// POST /api/v1/orders  (Idempotency-Key: <uuid>)
CreateOrderRequest = z.object({
  items: z.array(z.object({
    productId: z.string().uuid(),
    quantity: z.number().int().min(1).max(1000),
  })).min(1).max(50),
  paymentMethodToken:
z.enum(['pm_ok','pm_declined','pm_timeout','pm_slow_success','pm_error']),
  shippingAddress: AddressSchema,          // stored, not validated against carriers
  notes: z.string().max(500).optional(),
}).strict();
// 202 Accepted — saga continues asynchronously
CreateOrderResponse = { orderId, status: 'PENDING', totalMinor, currency, links: {
self, status } }

// POST /api/v1/inventory/adjustments
AdjustStockRequest = z.object({
  warehouseId: z.string().uuid(), productId: z.string().uuid(),
  delta: z.number().int().refine(v => v !== 0).min(-100000).max(100000),
  reason: z.enum(['RECEIPT','DAMAGE','COUNT_CORRECTION','RETURN']),
  note: z.string().min(3).max(500),
}).strict();
```

`POST /orders` returns **202** (not 201) because confirmation depends on the saga; clients poll `/orders/{id}/status` or receive a notification. If the saga completes within 2 s the gateway may return the final state in the same 202 body.

7.3 gRPC contract overview

Proto packages live in `proto/<service>/v1/*.proto`, linted and breaking-change-checked by Buf, and generated to TypeScript with `ts-proto` (`outputServices=grpc-js`).

Service	RPC	Kind	Caller(s)	Deadline	Retry?	Idempotency
`catalog.v1.CatalogService`	`GetProduct`	query	Gateway	500 ms	yes (2)	n/a
	`GetProducts` (batch ≤ 100 ids)	query	Order, Gateway	800 ms	yes (2)	n/a
	`ValidateProduct`	query	Inventory	500 ms	yes (2)	n/a
`inventory.v1.InventoryService`	`CheckAvailability`	query (advisory only)	Gateway, Order	500 ms	yes (2)	n/a
	`ReserveStock`	command	Order	2 s	yes (2)	`reservation_key = order_id`
	`ConfirmStock`	command	Order	2 s	yes (3)	`reservation_id` state check
	`ReleaseStock`	command	Order	2 s	yes (3)	`reservation_id` state check
	`GetInventory`, `GetReservation`	query	Gateway, Order	800 ms	yes (2)	n/a
`order.v1.OrderService`	`CreateOrder`	command	Gateway	3 s	**no** at gateway (client retries with same key)	`idempotency_key`
	`GetOrder`, `GetOrderStatus`	query	Gateway	800 ms	yes (2)	n/a
	`CancelOrder`	command	Gateway	3 s	no	`idempotency_key`
`payment.v1.PaymentService`	`ProcessPayment`	command	Order	3 s	yes (2), same key	`idempotency_key = order_id:attempt`

Service	RPC	Kind	Caller(s)	Deadline	Retry?	Idempotency
	`GetPaymentStatus`	query	Order (reconciler), Gateway	800 ms	yes (3)	n/a
	`ProcessRefund`	command	Order, Gateway	3 s	yes (2), same key	`refund_key`
`identity.v1.IdentityService`	`ValidateSession`	query	Gateway	300 ms	yes (1)	n/a
	`CheckPermission`	query	Order, Payment (high-risk)	300 ms	yes (1)	n/a
	`GetSigningKeys`	query	all (cached 10 min)	1 s	yes (3)	n/a

The Order/Gateway-facing RPCs that only mirror REST (`ListOrders`, `SearchProducts`, etc.) exist because the gateway holds no business logic; they are thin and carry no extra semantics.

Two further RPCs are required by later sections: `inventory.v1.RestockOrder(order_id)` (command, Order caller, 2 s, retry 3, idempotent by order ID; returns stock for a cancelled confirmed order, SAGA-007) and `identity.v1.ListUsersByPermission(permission, warehouse_id?)` (query, Notification caller, 800 ms; resolves low-stock alert recipients, Workflow C).

Representative proto (inventory):

```
syntax = "proto3";
package inventory.v1;

service InventoryService {
  rpc CheckAvailability(CheckAvailabilityRequest) returns
(CheckAvailabilityResponse);
  rpc ReserveStock(ReserveStockRequest) returns (ReserveStockResponse);
  rpc ConfirmStock(ConfirmStockRequest) returns (ConfirmStockResponse);
  rpc ReleaseStock(ReleaseStockRequest) returns (ReleaseStockResponse);
  rpc GetInventory(GetInventoryRequest) returns (GetInventoryResponse);
  rpc GetReservation(GetReservationRequest) returns (Reservation);
}

message ReserveLine { string product_id = 1; int32 quantity = 2; }
message ReserveStockRequest {
  string order_id = 1;            // also the idempotency key
  repeated ReserveLine lines = 2; // 1..50
  int32 ttl_seconds = 3;          // 60..3600, default 900
}
message Allocation { string product_id = 1; string warehouse_id = 2; int32 quantity =
3; }
```

```
message ReserveStockResponse {
  string reservation_id = 1;
  ReservationStatus status = 2;   // RESERVED
  repeated Allocation allocations = 3;
  google.protobuf.Timestamp expires_at = 4;
  bool replayed = 5;              // true when returned from idempotency record
}
message ReleaseStockRequest { string reservation_id = 1; ReleaseReason reason = 2; }
enum ReservationStatus { RESERVATION_STATUS_UNSPECIFIED = 0; RESERVED = 1; CONFIRMED
= 2; RELEASED = 3; EXPIRED = 4; }
enum ReleaseReason { RELEASE_REASON_UNSPECIFIED = 0; PAYMENT_FAILED = 1; CANCELLED =
2; EXPIRED_TTL = 3; SAGA_ABORTED = 4; }
```

Insufficient stock returns `FAILED_PRECONDITION` with a `google.rpc.ErrorInfo` detail `reason=INSUFFICIENT_STOCK` and per-line shortfall metadata.

7.4 gRPC engineering requirements

ID	Requirement	Priority
GRPC-001	`.proto` files are the single source; Buf `lint` (DEFAULT) and `breaking` (against `main`) run in CI; generated code is not hand-edited.	Must
GRPC-002	Every outgoing call sets a deadline from the table above; inbound handlers read the remaining deadline and propagate `min(remaining − 50 ms, own default)` downstream. A call with no deadline fails a lint rule.	Must
GRPC-003	Request messages validated in the handler with Zod mirrors (or `protovalidate`) before use; invalid → `INVALID_ARGUMENT` with field violations.	Must
GRPC-004	Metadata propagated: `x-correlation-id`, `traceparent`, `x-user-id`, `x-user-perms` (signed end-user JWT forwarded as `authorization`), `idempotency-key`.	Must
GRPC-005	Service-to-service authentication via mTLS with a dev CA (`step-ca` or generated by `make certs`); each server verifies the client certificate's SAN `spiffe://stockflow/<service>` against an allow-list per RPC. Plaintext gRPC is refused outside `NODE_ENV=test`.	Must
GRPC-006	End-user authorization re-checked in the owning service by verifying the forwarded JWT signature (keys from `GetSigningKeys`) and permissions; services do not trust `x-user-*` headers without a verified token.	Must
GRPC-007	Retries only for idempotent calls and only on `UNAVAILABLE`, `RESOURCE_EXHAUSTED` (with backoff) and `DEADLINE_EXCEEDED` for queries; exponential backoff 100 ms × 2^n with full jitter, cap 1 s, max attempts per table; total time bounded by the parent deadline.	Must

ID	Requirement	Priority
GRPC-008	Client-side circuit breaker per downstream (opossum): open after 50% failures over 20 calls in 10 s, half-open after 15 s; open circuit returns `UNAVAILABLE` fast.	Should
GRPC-009	Server and client interceptors emit structured logs (method, status, duration, correlation ID) and Prometheus histograms `grpc_server_handling_seconds` / `grpc_client_handling_seconds`.	Must
GRPC-010	Standard `grpc.health.v1` service implemented; `SERVING` mirrors readiness.	Must
GRPC-011	Max message size 4 MB; keepalive 30 s; graceful `tryShutdown` with 10 s drain.	Must
GRPC-012	Contract tests: each consumer's expectations (fixtures) run against the provider's real server in CI.	Must

Status-code policy:

gRPC status	Used when	Gateway HTTP	Retry by caller
`OK`	success, including idempotent replay	200/201/202	—
`INVALID_ARGUMENT`	schema/field validation	400	never
`UNAUTHENTICATED`	missing/invalid token or mTLS identity	401	never
`PERMISSION_DENIED`	lacks permission/ownership	403 (404 for ownership to avoid enumeration)	never
`NOT_FOUND`	entity absent	404	never
`ALREADY_EXISTS`	unique conflict (SKU, email) or idempotency key reused with different payload	409	never
`FAILED_PRECONDITION`	business rule: insufficient stock, invalid state transition	409	never
`ABORTED`	serialization failure / optimistic lock conflict	409 (after internal retry)	yes, internally (3)
`RESOURCE_EXHAUSTED`	rate/concurrency limit	429	yes with backoff

gRPC status	Used when	Gateway HTTP	Retry by caller
`DEADLINE_EXCEEDED`	deadline hit	504	queries only
`UNAVAILABLE`	downstream down, circuit open, shutting down	503	yes (idempotent only)
`INTERNAL` / `UNKNOWN`	bug	500	never

7.5 Timeout after a successful remote operation

A `DEADLINE_EXCEEDED` or `UNAVAILABLE` on a **command** means the outcome is **unknown**, not failed. Rules:

- **GRPC-013 (Must)** Every command carries a stable idempotency key, so the caller can safely re-send the identical request; the callee returns the stored original result (`replayed = true`) instead of executing twice.
- **GRPC-014 (Must)** On unknown outcome the Order saga moves to an `*_UNKNOWN` sub-state (e.g. `PAYMENT_PENDING_CONFIRMATION`), persists it, and resolves it by (1) retrying the same command with the same key up to the retry budget, then (2) querying `GetPaymentStatus` / `GetReservation`, then (3) handing over to the reconciler, which repeats the status query every 30 s for up to 15 min before raising an alert and leaving the order in `REQUIRES_ATTENTION`.
- **GRPC-015 (Must)** The caller never compensates an operation whose outcome is unknown until it has confirmed the outcome; compensations themselves are idempotent, so a compensation for an operation that never happened is a no-op (`ReleaseStock` on a non-existent or released reservation returns `OK` with `status=RELEASED`).
- Example: Payment captures the charge, then the response is lost. Order retries `ProcessPayment(key=order:1)`, Payment finds the existing `SUCCEEDED` row and returns it; the order proceeds to `CONFIRMED` with exactly one charge (AC-018).

gRPC request/response flow with deadline, retry and unknown outcome:

```
sequenceDiagram
  participant S as Order saga worker
  participant CI as client interceptors<br/>(deadline, metadata, retry, breaker,
metrics)
  participant SI as Payment server interceptors<br/>(mTLS identity, JWT, validation,
logging)
  participant H as ProcessPayment handler
  participant DB as payment_db
  S->>CI: processPayment({key: orderId:1, amount}, deadline = now+3s)
  CI->>SI: HTTP/2 call + x-correlation-id, traceparent, authorization,
idempotency-key
  SI->>SI: verify client cert SAN = order-svc, allow-listed for ProcessPayment
  SI->>H: validated request
  H->>DB: INSERT payments ... ON CONFLICT (idempotency_key) DO NOTHING
  alt new key
    H->>H: simulate gateway (scenario token)
```

```
  H->>DB: UPDATE status=SUCCEEDED + outbox payment.succeeded (one tx)
else existing key
  H->>DB: SELECT stored result (replayed=true)
end
alt response arrives within deadline
  H-->>CI: OK {status, paymentId, replayed}
  CI-->>S: result
else response lost / deadline hit
  CI-->>S: DEADLINE_EXCEEDED (outcome UNKNOWN)
  S->>S: persist PAYMENT_UNKNOWN
  S->>CI: retry same key (backoff + jitter, within budget)
  CI->>SI: processPayment(same key)
  SI->>H: validated retry
  H-->>S: OK replayed=true, SUCCEEDED (no second charge)
end
```

8. RabbitMQ topology and event schemas

All domain events flow through one durable topic exchange into per-consumer quorum queues, published from a transactional outbox with publisher confirms and consumed idempotently with tiered retries and a dead-letter queue — at-least-once delivery, never claimed as exactly-once.

8.1 Topology

Object	Name	Type / args	Purpose
Exchange	`domain.events`	topic, durable	All integration events
Exchange	`domain.retry`	direct, durable	Routes failed messages to delay queues
Exchange	`domain.dlx`	topic, durable	Terminal dead letters
Queue	`notification.q`	quorum, `x-dead-letter-exchange=domain.dlx`	Notification consumer
Queue	`audit.q`	quorum	Audit & reporting consumer
Queue	`order.inventory.q`	quorum	Order consumes reservation expiry/release
Queue	`order.payment.q`	quorum	Order consumes payment outcomes (reconciliation path)
Queue	`inventory.catalog.q`	quorum	Inventory consumes product archival
Queues	`<consumer>.retry.5s` / `.30s` / `.2m`	classic durable, `x-message-ttl`, `x-dead-letter-exchange=""`,	Delayed redelivery back to the main queue

Object	Name	Type / args	Purpose
		`x-dead-letter-routing-key=<consumer>.q`	
Queue	`<consumer>.dlq`	quorum, bound to `domain.dlx` with `<consumer>.#`	Poison / exhausted messages, retained 14 days (`x-message-ttl`)

Naming: exchanges `<scope>.<purpose>`; queues `<consumer-service>.<source>.q`; routing keys `<producer-context>.<aggregate>.<event>` in past tense, lowercase, dot-separated. Topology is declared in `deploy/compose/rabbitmq/definitions.json` (loaded at broker start) **and** asserted idempotently by each service on boot; mismatched arguments fail readiness.

8.2 Routing keys and bindings

Routing key	Producer	Consumers (queue)	Payload highlights
`catalog.product.created`	Catalog	audit	productId, sku, priceMinor
`catalog.product.updated`	Catalog	audit	productId, changed fields, version
`catalog.product.archived`	Catalog	inventory.catalog, audit	productId, sku
`inventory.stock.reserved`	Inventory	audit	reservationId, orderId, allocations
`inventory.stock.released`	Inventory	order.inventory, audit	reservationId, orderId, reason
`inventory.stock.confirmed`	Inventory	audit	reservationId, orderId
`inventory.stock.adjusted` / `.transferred`	Inventory	audit	movement ids, actor, reason
`inventory.stock.low`	Inventory	notification, audit	productId, warehouseId, available, threshold
`order.created`	Order	notification, audit	orderId, customerId, totalMinor
`order.confirmed`	Order	notification, audit	orderId, paymentId
`order.cancelled`	Order	notification, audit	orderId, reason, refundId?
`order.failed`	Order	notification, audit	orderId, failureCode

Routing key	Producer	Consumers (queue)	Payload highlights
`order.requires_attention`	Order	notification (admins), audit	orderId, stuckState
`payment.succeeded`	Payment	order.payment, audit	paymentId, orderId, amountMinor
`payment.failed`	Payment	order.payment, notification, audit	paymentId, orderId, declineCode
`payment.refunded`	Payment	notification, audit	refundId, paymentId, amountMinor
`identity.session.expired`	Identity	audit	sessionId, userId, expiredAt, detectedBy (`keyspace`/`reconciler`)
`identity.session.revoked`	Identity	notification (reuse only), audit	sessionId, userId, reason
`identity.user.registered` / `.role_changed`	Identity	audit	userId, roles
`audit.security.*`	Identity, Gateway	audit	login success/failure, rate-limit block

Bindings: `notification.q` ← `order.*`, `payment.failed`, `payment.refunded`, `inventory.stock.low`, `identity.session.revoked`; `audit.q` ← `#`.

```
flowchart LR
  P1[identity outbox-relay] --> X
  P2[catalog outbox-relay] --> X
  P3[inventory outbox-relay] --> X
  P4[order outbox-relay] --> X
  P5[payment outbox-relay] --> X
  X{{domain.events<br/>topic}}
  X -- "order.* payment.failed payment.refunded<br/>inventory.stock.low
identity.session.revoked" --> NQ[notification.q]
  X -- "#" --> AQ[audit.q]
  X -- "inventory.stock.released" --> OQ1[order.inventory.q]
  X -- "payment.*" --> OQ2[order.payment.q]
  X -- "catalog.product.archived" --> IQ[inventory.catalog.q]
  NQ --> NC[notification consumer]
  AQ --> AC[audit consumer]
  OQ1 --> OC[order consumer]
  OQ2 --> OC
  IQ --> IC[inventory consumer]
  NC -- "nack: transient" --> R{{domain.retry}}
  R --> RQ[notification.retry.5s / 30s / 2m<br/>TTL then DLX back]
  RQ --> NQ
```

```
  NC -- "attempts exhausted or poison" --> D{{domain.dlx}}
  D --> DLQ[notification.dlq]
```

8.3 Event envelope (v1)

```
EventEnvelope = z.object({
  eventId: z.string().uuid(),          // UUIDv7, stable across republish
  type: z.string(),                    // = routing key, e.g. 'order.confirmed'
  schemaVersion: z.number().int(),     // 1, 2, ... per type
  occurredAt: z.string().datetime(),
  producer: z.string(),                // 'order-svc@1.4.0'
  aggregateType: z.string(), aggregateId: z.string(),
  aggregateVersion: z.number().int(),  // for ordering/staleness checks
  correlationId: z.string().uuid(),
  causationId: z.string().uuid().optional(),
  actor: z.object({ type: z.enum(['user','service','system']), id: z.string() }),
  data: z.unknown(),                   // validated by per-type schema
});
```

AMQP properties: `persistent: true` (deliveryMode 2), `contentType: application/json`, `messageId = eventId`, `type`, `timestamp`, headers `x-correlation-id`, `traceparent`, `x-schema-version`. No PII beyond IDs; email addresses are never in events.

Schema evolution: additive changes keep the version; removing/renaming a field or changing semantics bumps `schemaVersion` and the producer dual-publishes v(n) and v(n+1) until all consumers migrate. Schemas live in `packages/contracts/events/<type>.v<n>.ts` with an AsyncAPI 3.0 document generated from them; contract tests validate producers' fixtures against the schemas.

8.4 Transactional outbox

```
CREATE TABLE outbox (
  id            uuid PRIMARY KEY,            -- = eventId
  aggregate_type text NOT NULL, aggregate_id text NOT NULL,
  routing_key   text NOT NULL,
  payload       jsonb NOT NULL,              -- full envelope
  headers       jsonb NOT NULL DEFAULT '{}',
  created_at    timestamptz NOT NULL DEFAULT now(),
  published_at  timestamptz,
  attempts      int NOT NULL DEFAULT 0,
  next_attempt_at timestamptz NOT NULL DEFAULT now(),
  last_error    text
);
CREATE INDEX outbox_unpublished_idx ON outbox (next_attempt_at) WHERE published_at IS
NULL;
```

Relay algorithm (per service, 1+ replicas):

1. `BEGIN; SELECT … FROM outbox WHERE published_at IS NULL AND next_attempt_at <= now() ORDER BY created_at LIMIT 100 FOR UPDATE SKIP LOCKED;`
2. Publish each row on a **ConfirmChannel** with `mandatory: true`; await `waitForConfirms()` (timeout 5 s).
3. Acked rows → `UPDATE outbox SET published_at = now()`; nacked/returned/timed-out rows → `attempts += 1`, `next_attempt_at = now() + backoff(attempts)` (1 s → 5 min cap), `last_error`. `COMMIT`.

4. Poll every 200 ms when the last batch was full, else 1 s; also `LISTEN outbox_new` (trigger `NOTIFY`) for low latency.
5. A crash between broker confirm and `COMMIT` republishes the same `eventId` — consumers deduplicate. Ordering is per-aggregate best-effort (`ORDER BY created_at`); consumers use `aggregateVersion` where order matters.
6. Published rows are deleted after 7 days by a housekeeping job.

```
sequenceDiagram
  participant API as Order handler
  participant DB as order_db
  participant R as outbox-relay
  participant MQ as RabbitMQ
  participant C as Notification consumer
  participant NDB as notification_db
  API->>DB: BEGIN; UPDATE orders SET status='CONFIRMED'; INSERT
outbox(order.confirmed); COMMIT
  loop every 200ms-1s
    R->>DB: SELECT ... FOR UPDATE SKIP LOCKED
    R->>MQ: publish (persistent, confirm channel)
    alt broker confirms
      MQ-->>R: basic.ack
      R->>DB: SET published_at=now(); COMMIT
    else nack / timeout / broker down
      R->>DB: attempts+1, next_attempt_at=backoff; COMMIT
    end
  end
  MQ->>C: deliver (may be duplicate)
  C->>NDB: BEGIN; INSERT processed_events(consumer,event_id) ON CONFLICT DO NOTHING
  alt first time
    C->>NDB: INSERT notification; COMMIT
    C->>MQ: ack
  else duplicate (0 rows inserted)
    C->>NDB: ROLLBACK
    C->>MQ: ack (no side effect)
  end
```

8.5 Messaging requirements

ID	Requirement	Priority
MQ-001	Domain events are published only via the outbox; business code never calls `channel.publish` directly (enforced by a Semgrep custom rule).	Must
MQ-002	Business change and outbox insert occur in the same PostgreSQL transaction.	Must
MQ-003	Relay uses publisher confirms and `mandatory` routing; unroutable messages (returned) are treated as failures and alerted.	Must
MQ-004	All exchanges and queues durable; messages persistent; consumer queues are quorum queues.	Must

ID	Requirement	Priority
MQ-005	Consumers use manual ack, `prefetch` 20 (configurable), and ack only after their DB transaction commits.	Must
MQ-006	Idempotent consumption: `processed_events(consumer, event_id)` PK inserted in the same transaction as the side effect; duplicates are acked without effect. Retained 30 days.	Must
MQ-007	Error classification: **transient** (DB/Redis unavailable, timeouts) → retry via 5 s, 30 s, 2 m delay queues, then 2 m repeated until 10 total attempts; **permanent** (schema invalid, unknown version, business rule violation) → straight to DLQ. Attempt count tracked in `x-retry-count` header.	Must
MQ-008	DLQ messages keep original headers + `x-death`, `x-last-error`, `x-failed-at`; `dlq_messages` gauge alerts when > 0 for 5 min.	Must
MQ-009	Replay CLI `pnpm mq:replay --queue notification.dlq --filter type=order.confirmed --limit 50 [--dry-run]` republishes to the original routing key with the same `eventId` (dedup makes replay safe); every replay is audited.	Should
MQ-010	Consumers validate envelope and `data` against the schema for `type+schemaVersion` before processing; invalid → DLQ with reason `SCHEMA_INVALID`.	Must
MQ-011	Broker unavailability never fails a user request whose business write committed: events wait in the outbox. Relay reconnects with backoff (1 s → 30 s).	Must
MQ-012	Consumers reconnect automatically on channel/connection close; in-flight unacked messages are redelivered by the broker.	Must
MQ-013	Metrics: `outbox_backlog`, `outbox_oldest_unpublished_seconds`, `mq_consume_total{result}`, `mq_retry_total`, `mq_dlq_total`, queue depth via RabbitMQ Prometheus plugin.	Must
MQ-014	Outbox and idempotent consumers are used only where durable delivery matters (all routing keys above). Purely ephemeral signals (cache invalidation fan-out) use Redis pub/sub without outbox.	Must

8.6 Required asynchronous workflows

- **Workflow A — Order lifecycle:** Section 11.3; events `order.created` → `inventory.stock.reserved` → `payment.succeeded` → `inventory.stock.confirmed` → `order.confirmed` → notification.

- **Workflow B — Payment failure:** Section 11.4; `payment.failed` → order `FAILED` → `ReleaseStock` → `inventory.stock.released` → `order.failed` → notification; the saga reconciler detects orders in `COMPENSATING` older than 2 min.
- **Workflow C — Low stock:** any balance change in Inventory evaluates the threshold inside the same transaction; on downward crossing it sets `low_stock_state.alerted = true` and writes `inventory.stock.low` to the outbox; Notification fans out to all users with `inventory_manager` (recipient list resolved via `Identity.ListUsersByPermission`, cached 5 min).
- **Workflow D — Session lifecycle:** Section 9.5; `identity.session.expired|revoked` → audit row + `daily_summaries` counters updated idempotently by `event_id`.

9. Redis: caching, sessions and token lifecycle

Redis is a performance and coordination layer, never the system of record: PostgreSQL holds durable session, token and inventory truth, so a Redis flush or outage degrades latency but never correctness or security.

9.1 Use cases and key namespaces

Namespace	Owner	Purpose	Example key	TTL
`cat:`	Catalog	Product / category cache	`cat:product:v1:{productId}`	10 min ± 10% jitter
`cat:`	Catalog	Category tree	`cat:category-tree:v1`	30 min
`inv:`	Inventory	Warehouse metadata cache (not balances)	`inv:warehouse:v1:{id}`	30 min
`sess:`	Identity	Hot session state	`sess:{sid}` (hash)	refresh idle TTL + 5 min
`sess:exp:`	Identity	Expiry shadow key (triggers notification)	`sess:exp:{sid}`	refresh idle TTL (7 d), capped at absolute expiry
`usess:`	Identity	Sessions per user	`usess:{userId}` (sorted set, score=createdAt)	none (pruned on change)
`jwks:`	each service (own)	Cached signing keys	`jwks:{svc}`	10 min
`gw:rl:`	Gateway	Rate-limit windows	`gw:rl:user:{id}:{window}`	window length
`idem:`	each service (own)	Short-lived idempotency fast path (PG is authoritative)	`idem:order:{key}`	24 h

Namespace	Owner	Purpose	Example key	TTL
`lock:`	each service (own)	Job coordination	`lock:job:inventory:reservation-sweeper`	60 s, renewed

Each service uses a key prefix it alone owns and a Redis ACL user restricted to that prefix (`~cat:*` etc.); no service reads another's keys. Local dev runs one Redis; production may split cache and session instances (eviction `allkeys-lru` for cache, `noeviction` for sessions).

9.2 Cache-aside

- **REDIS-001 (Must)** Read path: `GET key` → hit: return; miss: load from PostgreSQL, `SET key value EX ttl`, return. Not-found results are cached as a tombstone for 30 s to blunt enumeration.
- **REDIS-002 (Must)** Write path: commit to PostgreSQL first, then `DEL key` (and dependent keys, e.g. category tree). If `DEL` fails, the TTL bounds staleness; the failure is logged and counted.
- **REDIS-003 (Must)** Stale-data policy: catalog data may be up to 10 min stale on read; **order pricing never trusts the cache** — `Catalog.GetProducts` called by Order bypasses the cache (`consistency: STRONG` field) so price snapshots and status checks come from PostgreSQL.
- **REDIS-004 (Must)** Inventory balances are never cached for decisions; `CheckAvailability` is advisory and the reservation's conditional `UPDATE` is authoritative.
- **REDIS-005 (Should)** Stampede mitigation for hot keys: single-flight via `SET lock:cache:{key} token NX PX 3000`; losers poll the key every 50 ms up to 300 ms, then read PostgreSQL directly. TTL jitter prevents synchronized expiry.
- **REDIS-006 (Must)** Redis unavailable: the cache client wraps calls in a 50 ms timeout and a circuit breaker; on failure reads go straight to PostgreSQL, writes skip invalidation, `/readyz` stays **ready** (Redis is a soft dependency for Catalog), and `cache_errors_total` increments.
- **REDIS-007 (Must)** Metrics `cache_requests_total{result=hit|miss|error, cache}` and hit-rate panel; target ≥ 85% hit rate on product detail at steady state.
- Values are versioned JSON (`v1` in key) so a schema change rolls keys instead of deserializing old shapes.

9.3 Sessions

A **session** = one login on one device. PostgreSQL `sessions` is durable truth (history, audit, reconciliation); Redis `sess:{sid}` is the hot copy used on every request.

```
sess:{sid} = HASH { userId, familyId, status: ACTIVE|REVOKED, createdAt, lastSeenAt,
                    idleExpiresAt, absoluteExpiresAt, ipPrefix, userAgentHash }
```

- **REDIS-008 (Must)** Login creates the PG row and Redis hash in that order; Redis write failure does not fail login (lazy rewarm on next validation).
- **REDIS-009 (Must)** Multiple concurrent sessions allowed, default cap 10 per user; the oldest is revoked when exceeded (configurable).
- **REDIS-010 (Must)** Idle timeout 7 days (sliding, extended on refresh), absolute timeout 30 days; both enforced in `ValidateSession` and refresh.

- **REDIS-011 (Must)** Logout / revoke: PG `status = REVOKED` + outbox event in one transaction, then Redis `HSET status REVOKED` and `DEL sess:exp:{sid}` so a revoked session never also emits `expired`.
- **REDIS-012 (Must)** `ValidateSession(sid)`: Redis hash → if missing, PostgreSQL → rewarm. Redis down → PostgreSQL with a 1 s in-process cache. PostgreSQL also down → `UNAVAILABLE` and the gateway returns 503 (**fail closed**; never allow an unverified session).
- **REDIS-013 (Must)** The gateway caches positive `ValidateSession` results in-process for at most 1 s; revocation therefore takes effect within ~1 s (G-4).
- `lastSeenAt` is updated at most once per minute per session to bound write volume.

9.4 Token lifecycle

Token	Format	Lifetime	Stored server-side as
Access	JWT, ES256, `kid` header; claims `iss`, `aud`, `sub`, `sid`, `perms`, `wh`, `iat`, `nbf`, `exp`, `jti`	10 min	not stored (stateless) — liveness via `sid` check
Refresh	Opaque: `base64url(sid).base64url(32 random bytes)`	7 d idle / 30 d family absolute	SHA-256 hash in PG `refresh_tokens` (never the raw value)

- **REDIS-014 (Must)** JWT verification uses `jose` with an algorithm allow-list `['ES256']`, required `iss`/`aud`/`exp`, clock tolerance 30 s; `alg: none`, HS/RS confusion and missing `kid` are rejected. Signing keys rotate every 30 days with a 2-key overlap; private keys come from the secret store, never the repo.
- **REDIS-015 (Must)** Rotation in one PG transaction: `UPDATE refresh_tokens SET used_at = now(), replaced_by = $new WHERE token_hash = $h AND used_at IS NULL AND revoked_at IS NULL AND expires_at > now() RETURNING family_id` → insert new token (generation + 1).
- **REDIS-016 (Must)** Reuse detection: if the hash exists but `used_at IS NOT NULL`, and the reuse is outside a 10 s grace window (concurrent-tab race), revoke the whole family and its session, emit `identity.session.revoked{reason: token_reuse}` and `audit.security.refresh_reuse`, return 401. Inside the grace window, return the already-issued successor once.
- **REDIS-017 (Must)** Revoked session ⇒ its refresh family is revoked (`revoked_at`) and its access tokens die at the next `ValidateSession` (≤ 1 s), not at `exp`.
- **REDIS-018 (Must)** Refresh cookie: `HttpOnly; Secure; SameSite=Strict; Path=/api/v1/auth`; access token returned in the body and held in memory by the SPA (never `localStorage`).

9.5 Expiry-event processing

Redis keyspace notifications are fire-and-forget: they reach only currently connected subscribers, are lost during disconnects, and fire when Redis actually evicts the key (which can lag the TTL). They are a **latency optimisation**; the reconciler guarantees correctness.

- **REDIS-019 (Must)** Redis runs with `notify-keyspace-events Ex`; Identity holds a dedicated subscriber connection on `__keyevent@0__:expired` and handles keys matching `sess:exp:*`.
- **REDIS-020 (Must)** The shadow-key pattern is used because an expired key's data is gone: `sess:exp:{sid}` expires first; the handler reads `sess:{sid}` (which lives 5 min longer) or falls back to PG.
- **REDIS-021 (Must)** Handler, idempotent across replicas: `UPDATE sessions SET status='EXPIRED', ended_at=now(), end_detected_by='keyspace' WHERE id=$sid AND status='ACTIVE' RETURNING *` + outbox `identity.session.expired` in the same transaction; zero rows ⇒ already handled, do nothing.
- **REDIS-022 (Must)** Reconciler every 5 min (leader via `lock:job:identity:session-reconciler`): same UPDATE for `status='ACTIVE' AND LEAST(idle_expires_at, absolute_expires_at) < now() - interval '1 minute'`, `end_detected_by='reconciler'`, batches of 500. Metric `session_expiry_reconciled_total` shows how many notifications were missed.
- **REDIS-023 (Must)** Session cleanup: expired `refresh_tokens` rows purged after 30 days; `sessions` rows retained 1 year for audit, then archived.

```
sequenceDiagram
  participant R as Redis
  participant ID as identity-svc (subscriber)
  participant REC as identity reconciler (every 5 min)
  participant DB as identity_db
  participant MQ as RabbitMQ
  participant AU as audit-svc
  R--)ID: __keyevent@0__:expired  sess:exp:{sid}
  ID->>R: HGETALL sess:{sid}  (still alive +5 min)
  ID->>DB: BEGIN; UPDATE sessions SET status=EXPIRED WHERE id=sid AND status=ACTIVE;
INSERT outbox; COMMIT
  alt 1 row updated
    DB-->>MQ: relay publishes identity.session.expired (detectedBy=keyspace)
  else 0 rows (other replica won / already revoked)
    ID->>ID: no-op
  end
  Note over R,ID: Notification lost (subscriber disconnected, Redis restart)
  REC->>DB: UPDATE sessions SET status=EXPIRED WHERE status=ACTIVE AND expires_at <
now()-1min RETURNING; INSERT outbox per row
  DB-->>MQ: relay publishes identity.session.expired (detectedBy=reconciler)
  MQ->>AU: deliver (possibly duplicate)
  AU->>AU: INSERT processed_events ON CONFLICT DO NOTHING; audit row; UPSERT
daily_summaries counters
```

9.6 Distributed coordination

- **REDIS-024 (Should)** Background jobs (reservation sweeper, saga reconciler, session reconciler, outbox housekeeping) take a leader lock `SET lock:job:{svc}:{job} {token} NX PX 60000`, renew every 20 s via Lua compare-and-`PEXPIRE`, and release via Lua compare-and-`DEL`.
- **REDIS-025 (Must)** Locks only **reduce duplicate work**; every job remains correct if two instances run concurrently (conditional `UPDATE … WHERE status = …`, `SKIP LOCKED`). Lock loss (Redis down, GC pause) is therefore safe.

- **REDIS-026 (Must)** Inventory correctness never depends on a Redis lock (no Redlock); it depends on PostgreSQL row locks and `CHECK` constraints (Section 10.3).
- **REDIS-027 (Must)** Gateway rate limiting uses an atomic Lua sliding-window script; if Redis is unavailable, rate limiting falls back to an in-process token bucket per instance (fail open for rate limiting only, logged).

10. PostgreSQL schemas and consistency rules

Each service owns one database; inventory correctness rests on PostgreSQL row locks, a generated `available` column and `CHECK (available >= 0)`, so overselling is impossible even if application code is wrong.

10.1 Database requirements

ID	Requirement	Priority
DB-001	One database and one login role per service; role has DML on its own schema only, no `SUPERUSER`, no `CREATEDB`. A separate migration role owns DDL.	Must
DB-002	Schema changes only through versioned node-pg-migrate migrations in `services/<svc>/migrations`, run by a one-shot `migrate` container before the service starts; migrations are forward-only and backward-compatible for one release (expand → migrate → contract).	Must
DB-003	All SQL parameterized (`$1`, Kysely); string concatenation into SQL is a Semgrep ERROR.	Must
DB-004	IDs are UUIDv7 generated in the application (time-ordered, index-friendly); money is `bigint` minor units + `char(3)` currency; timestamps `timestamptz` in UTC.	Must
DB-005	Every table has `created_at`; mutable tables have `updated_at` and `version int` for optimistic locking.	Must
DB-006	Default isolation READ COMMITTED; inventory reservation and transfer use explicit `SELECT … FOR UPDATE` in a deterministic lock order (product_id, warehouse_id) to avoid deadlocks; serialization/deadlock errors (`40001`, `40P01`) are retried up to 3 times with jitter.	Must
DB-007	Statement timeout 5 s, idle-in-transaction timeout 10 s, lock timeout 2 s per role.	Must
DB-008	Connection pools sized per service (default max 10 per replica) and documented against `max_connections`.	Must
DB-009	Cursor (keyset) pagination on `(sort_key, id)` with matching composite indexes; no `OFFSET` beyond page 1.	Must

ID	Requirement	Priority
DB-010	Soft delete via `status = ARCHIVED` for products, categories, warehouses; hard delete forbidden where referenced by ledger/history.	Must
DB-011	Retention: audit events 1 y online then archive; processed_events 30 d; published outbox 7 d; notifications 180 d; idempotency records 24 h (orders keep their key forever on the row).	Should
DB-012	Nightly `pg_dump` per database in Compose (`backup` profile); restore procedure in runbook.	Could

10.2 Schemas by service (key tables)

identity_db

```
CREATE TABLE users (
  id uuid PRIMARY KEY, email citext NOT NULL UNIQUE, password_hash text NOT NULL,
  status text NOT NULL CHECK (status IN ('ACTIVE','LOCKED','DISABLED')),
  failed_logins int NOT NULL DEFAULT 0, locked_until timestamptz,
  created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT NULL
DEFAULT now(), version int NOT NULL DEFAULT 1);
CREATE TABLE roles (id uuid PRIMARY KEY, name text NOT NULL UNIQUE);
CREATE TABLE permissions (id uuid PRIMARY KEY, code text NOT NULL UNIQUE);  --
'inventory:adjust'
CREATE TABLE role_permissions (role_id uuid REFERENCES roles, permission_id uuid
REFERENCES permissions, PRIMARY KEY (role_id, permission_id));
CREATE TABLE user_roles (user_id uuid REFERENCES users, role_id uuid REFERENCES
roles, PRIMARY KEY (user_id, role_id));
CREATE TABLE user_warehouses (user_id uuid REFERENCES users, warehouse_id uuid NOT
NULL, PRIMARY KEY (user_id, warehouse_id)); -- warehouse_id: opaque ref
CREATE TABLE sessions (
  id uuid PRIMARY KEY, user_id uuid NOT NULL REFERENCES users,
  status text NOT NULL CHECK (status IN ('ACTIVE','REVOKED','EXPIRED')),
  ip_prefix inet, user_agent_hash text, created_at timestamptz NOT NULL DEFAULT
now(),
  last_seen_at timestamptz, idle_expires_at timestamptz NOT NULL, absolute_expires_at
timestamptz NOT NULL,
  ended_at timestamptz, end_reason text, end_detected_by text CHECK (end_detected_by
IN ('user','admin','reuse','keyspace','reconciler')));
CREATE INDEX sessions_active_expiry_idx ON sessions (LEAST(idle_expires_at,
absolute_expires_at)) WHERE status = 'ACTIVE';
CREATE INDEX sessions_user_idx ON sessions (user_id, created_at DESC);
CREATE TABLE refresh_tokens (
  token_hash bytea PRIMARY KEY, family_id uuid NOT NULL, session_id uuid NOT NULL
REFERENCES sessions,
  generation int NOT NULL, issued_at timestamptz NOT NULL DEFAULT now(), expires_at
timestamptz NOT NULL,
  used_at timestamptz, replaced_by bytea, revoked_at timestamptz,
  UNIQUE (family_id, generation));
-- + outbox
```

catalog_db

```
CREATE TABLE categories (id uuid PRIMARY KEY, parent_id uuid REFERENCES categories,
name text NOT NULL,
  path ltree NOT NULL, depth int NOT NULL CHECK (depth BETWEEN 1 AND 4), status text
NOT NULL DEFAULT 'ACTIVE',
  UNIQUE (parent_id, name));
CREATE TABLE products (
  id uuid PRIMARY KEY, sku text NOT NULL, name text NOT NULL CHECK (length(name)
BETWEEN 2 AND 200),
  description text, category_id uuid NOT NULL REFERENCES categories,
  price_minor bigint NOT NULL CHECK (price_minor > 0), currency char(3) NOT NULL,
  status text NOT NULL CHECK (status IN ('DRAFT','ACTIVE','ARCHIVED')),
  low_stock_threshold int NOT NULL DEFAULT 10 CHECK (low_stock_threshold >= 0),
  search tsvector GENERATED ALWAYS AS (to_tsvector('simple', sku || ' ' || name))
STORED,
  created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT NULL
DEFAULT now(), version int NOT NULL DEFAULT 1);
CREATE UNIQUE INDEX products_sku_uq ON products (upper(sku));
CREATE INDEX products_search_idx ON products USING gin (search);
CREATE INDEX products_cat_created_idx ON products (category_id, created_at DESC, id);
CREATE TABLE product_media (id uuid PRIMARY KEY, product_id uuid NOT NULL REFERENCES
products, object_key text NOT NULL UNIQUE,
  mime_type text NOT NULL CHECK (mime_type IN
('image/jpeg','image/png','image/webp','application/pdf')),
  size_bytes int NOT NULL CHECK (size_bytes BETWEEN 1 AND 10485760), status text NOT
NULL CHECK (status IN ('PENDING','READY','REJECTED')),
  sha256 bytea, created_at timestamptz NOT NULL DEFAULT now());
-- + outbox
```

inventory_db

```
CREATE TABLE warehouses (id uuid PRIMARY KEY, code text NOT NULL UNIQUE, name text
NOT NULL, address jsonb NOT NULL,
  priority int NOT NULL DEFAULT 100, status text NOT NULL CHECK (status IN
('ACTIVE','INACTIVE')), version int NOT NULL DEFAULT 1);
CREATE TABLE inventory_balances (
  product_id uuid NOT NULL, warehouse_id uuid NOT NULL REFERENCES warehouses,
  on_hand int NOT NULL CHECK (on_hand >= 0),
  reserved int NOT NULL DEFAULT 0 CHECK (reserved >= 0),
  available int GENERATED ALWAYS AS (on_hand - reserved) STORED,
  low_stock_threshold int NOT NULL, low_stock_alerted boolean NOT NULL DEFAULT false,
  version int NOT NULL DEFAULT 1, updated_at timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (product_id, warehouse_id),
  CONSTRAINT available_non_negative CHECK (on_hand - reserved >= 0));
CREATE TABLE reservations (
  id uuid PRIMARY KEY, order_id uuid NOT NULL UNIQUE,            -- idempotency
  status text NOT NULL CHECK (status IN
('RESERVED','CONFIRMED','RELEASED','EXPIRED')),
  expires_at timestamptz NOT NULL, created_at timestamptz NOT NULL DEFAULT now(),
updated_at timestamptz NOT NULL DEFAULT now(),
  release_reason text);
CREATE INDEX reservations_expiry_idx ON reservations (expires_at) WHERE status =
'RESERVED';
CREATE TABLE reservation_lines (reservation_id uuid REFERENCES reservations,
product_id uuid NOT NULL, warehouse_id uuid NOT NULL,
  quantity int NOT NULL CHECK (quantity > 0), PRIMARY KEY (reservation_id,
product_id, warehouse_id));
```

```
CREATE TABLE movements (
  id uuid PRIMARY KEY, product_id uuid NOT NULL, warehouse_id uuid NOT NULL,
  type text NOT NULL CHECK (type IN
('RECEIPT','SALE','DAMAGE','COUNT_CORRECTION','RETURN','TRANSFER_OUT','TRANSFER_IN'))
,
  qty_delta int NOT NULL CHECK (qty_delta <> 0),        -- change to on_hand
  reference_type text, reference_id uuid, reason_note text,
  actor_id text NOT NULL, correlation_id uuid NOT NULL, created_at timestamptz NOT
NULL DEFAULT now());
CREATE INDEX movements_product_wh_idx ON movements (product_id, warehouse_id,
created_at DESC);
REVOKE UPDATE, DELETE ON movements FROM inventory_app;   -- append-only
-- + outbox, processed_events, idempotency_keys(key, request_hash, response,
created_at)
```

order_db

```
CREATE TABLE orders (
  id uuid PRIMARY KEY, customer_id uuid NOT NULL, idempotency_key uuid NOT NULL,
request_hash bytea NOT NULL,
  status text NOT NULL CHECK (status IN
('PENDING','RESERVED','PAYMENT_PENDING','PAYMENT_UNKNOWN','PAID',

'CONFIRMED','FULFILLED','COMPENSATING','FAILED','CANCELLATION_REQUESTED','CANCELLED',
'REQUIRES_ATTENTION')),
  total_minor bigint NOT NULL CHECK (total_minor > 0), currency char(3) NOT NULL,
  reservation_id uuid, payment_id uuid, failure_code text,
  shipping_address jsonb NOT NULL, notes text,
  created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT NULL
DEFAULT now(), version int NOT NULL DEFAULT 1,
  UNIQUE (customer_id, idempotency_key));
CREATE INDEX orders_status_updated_idx ON orders (status, updated_at);   --
reconciler
CREATE INDEX orders_customer_created_idx ON orders (customer_id, created_at DESC,
id);
CREATE TABLE order_items (order_id uuid REFERENCES orders, line_no int, product_id
uuid NOT NULL, sku text NOT NULL,
  product_name text NOT NULL, unit_price_minor bigint NOT NULL CHECK
(unit_price_minor > 0),
  quantity int NOT NULL CHECK (quantity BETWEEN 1 AND 1000),
  line_total_minor bigint GENERATED ALWAYS AS (unit_price_minor * quantity) STORED,
PRIMARY KEY (order_id, line_no));
CREATE TABLE order_status_history (id bigserial PRIMARY KEY, order_id uuid NOT NULL
REFERENCES orders,
  from_status text, to_status text NOT NULL, reason text, actor_id text NOT NULL,
correlation_id uuid NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now());
CREATE TABLE saga_state (order_id uuid PRIMARY KEY REFERENCES orders, step text NOT
NULL, attempt int NOT NULL DEFAULT 0,
  next_check_at timestamptz NOT NULL, last_error text, updated_at timestamptz NOT
NULL DEFAULT now());
-- + outbox, processed_events
```

payment_db

```
CREATE TABLE payments (
  id uuid PRIMARY KEY, order_id uuid NOT NULL, idempotency_key text NOT NULL UNIQUE,
```

```
  amount_minor bigint NOT NULL CHECK (amount_minor > 0), currency char(3) NOT NULL,
  status text NOT NULL CHECK (status IN ('PROCESSING','SUCCEEDED','FAILED')),
  method_token text NOT NULL CHECK (method_token ~ '^pm_[a-z_]+$'), decline_code
text,
  refunded_minor bigint NOT NULL DEFAULT 0, CONSTRAINT refund_le_amount CHECK
(refunded_minor <= amount_minor),
  created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT NULL
DEFAULT now());
CREATE INDEX payments_order_idx ON payments (order_id);
CREATE TABLE payment_attempts (id uuid PRIMARY KEY, payment_id uuid REFERENCES
payments, outcome text NOT NULL,
  latency_ms int NOT NULL, simulated_scenario text NOT NULL, created_at timestamptz
NOT NULL DEFAULT now());
CREATE TABLE refunds (id uuid PRIMARY KEY, payment_id uuid NOT NULL REFERENCES
payments, refund_key text NOT NULL UNIQUE,
  amount_minor bigint NOT NULL CHECK (amount_minor > 0), status text NOT NULL CHECK
(status IN ('SUCCEEDED','FAILED')),
  reason text NOT NULL, actor_id text NOT NULL, created_at timestamptz NOT NULL
DEFAULT now());
-- + outbox
```

notification_db — `notifications(id, recipient_id, type, title, body, data jsonb, source_event_id uuid, read_at, created_at, UNIQUE (source_event_id, recipient_id))`, index `(recipient_id, read_at NULLS FIRST, created_at DESC)`; `processed_events`.

audit_db — `audit_events(id, event_id uuid UNIQUE, type, actor_type, actor_id, resource_type, resource_id, correlation_id, occurred_at, data jsonb)` partitioned monthly by `occurred_at`, app role INSERT/SELECT only; `daily_summaries(date, metric, value bigint, PRIMARY KEY (date, metric))`; `summary_contributions(event_id, metric, PRIMARY KEY (event_id, metric))` so counters increment once per event.

Shared shape: `processed_events(consumer text, event_id uuid, processed_at timestamptz, PRIMARY KEY (consumer, event_id))`.

10.3 Inventory consistency rules

- **DB-013 (Must) Invariants**: `on_hand >= 0`, `reserved >= 0`, `on_hand - reserved >= 0` per (product, warehouse); `on_hand = SUM(movements.qty_delta)`; `reserved = SUM(reservation_lines.quantity)` over `RESERVED` reservations. A nightly invariant check job reports drift as a metric and alert.
- **DB-014 (Must) Reserve** (single transaction):
    a. `INSERT INTO reservations (... order_id ...) ON CONFLICT (order_id) DO NOTHING RETURNING id` — conflict ⇒ return existing reservation (idempotent replay).
    b. `SELECT ... FROM inventory_balances b JOIN warehouses w ... WHERE product_id = ANY($1) AND w.status='ACTIVE' ORDER BY product_id, warehouse_id FOR UPDATE OF b` (deterministic order).
    c. Allocate in memory per line by warehouse priority, then highest `available`.
    d. For each allocation: `UPDATE inventory_balances SET reserved = reserved + $q, version = version + 1 WHERE product_id=$p AND warehouse_id=$w AND on_hand - reserved >= $q` — 0 rows ⇒ abort with `INSUFFICIENT_STOCK`.

    e. Insert `reservation_lines`, evaluate low-stock, insert outbox `inventory.stock.reserved`; `COMMIT`.
- **DB-015 (Must) Confirm**: `UPDATE reservations SET status='CONFIRMED' WHERE id=$1 AND status='RESERVED'`; if 1 row, decrement `on_hand` and `reserved` by line quantity, insert `SALE` movements; if 0 rows and already `CONFIRMED` ⇒ OK (replay); if `RELEASED/EXPIRED` ⇒ `FAILED_PRECONDITION RESERVATION_NOT_ACTIVE`.
- **DB-016 (Must) Release / expire**: `UPDATE reservations SET status=$target WHERE id=$1 AND status='RESERVED'` then decrement `reserved`; already released/expired ⇒ OK no-op; `CONFIRMED` ⇒ `FAILED_PRECONDITION` (a confirmed sale is reversed with a `RETURN` adjustment, not a release).
- **DB-017 (Must) Sweeper** every 30 s: `SELECT id FROM reservations WHERE status='RESERVED' AND expires_at < now() LIMIT 200 FOR UPDATE SKIP LOCKED` → expire each as DB-016 and emit `inventory.stock.released{reason: EXPIRED_TTL}`.
- **DB-018 (Must) Adjust/transfer** lock both balance rows in (product, warehouse) order; transfer writes `TRANSFER_OUT` and `TRANSFER_IN` movements with a shared `reference_id`.

Concurrent reservation of the last units:

```
sequenceDiagram
  participant O1 as Order A (wants 2)
  participant O2 as Order B (wants 2)
  participant INV as inventory-svc
  participant DB as inventory_db (on_hand 3, reserved 0)
  par
    O1->>INV: ReserveStock(order A)
    O2->>INV: ReserveStock(order B)
  end
  INV->>DB: Tx A: SELECT ... FOR UPDATE (row locked)
  INV->>DB: Tx B: SELECT ... FOR UPDATE (waits on lock)
  INV->>DB: Tx A: UPDATE reserved=reserved+2 WHERE on_hand-reserved >= 2  -> 1 row
  INV->>DB: Tx A: COMMIT (available = 1)
  DB-->>INV: Tx B lock acquired, sees available = 1
  INV->>DB: Tx B: UPDATE ... WHERE on_hand-reserved >= 2  -> 0 rows
  INV->>DB: Tx B: ROLLBACK
  INV-->>O1: OK reservationId
  INV-->>O2: FAILED_PRECONDITION INSUFFICIENT_STOCK
  Note over DB: CHECK (on_hand - reserved >= 0) is the last line of defence if code
skips the WHERE guard
```

11. Order orchestration and compensation

The Order Service runs an orchestrated, persisted saga: every step transition is a single PostgreSQL transaction (status + history + saga_state + outbox), every forward step has an idempotent compensation, and a reconciler drives any order that stops moving.

11.1 Saga requirements

ID	Requirement	Priority
SAGA-001	Orchestration (not choreography): the Order Service decides every next step; Inventory and Payment never call Order.	Must
SAGA-002	`CreateOrder` persists `PENDING` + `saga_state(step=RESERVE)` + outbox `order.created` in one transaction and returns 202; the saga executes asynchronously in the order worker.	Must
SAGA-003	The saga worker claims due sagas with `SELECT … FROM saga_state WHERE next_check_at <= now() FOR UPDATE SKIP LOCKED LIMIT 50`, so multiple replicas never run the same order concurrently.	Must
SAGA-004	Each remote call uses the idempotency key defined in Section 7.3; the step is persisted **before** the call (intent) and the result **after** (outcome).	Must
SAGA-005	Transitions are validated by a pure `transition(state, event) → state	Error` function with an exhaustive table; illegal transitions throw and are covered by unit tests.	Must
SAGA-006	Unknown outcomes enter `PAYMENT_UNKNOWN` (or retry the reserve/confirm step) and are resolved per GRPC-014; no compensation before the outcome is known.	Must
SAGA-007	Compensations: `ReleaseStock` for a reservation, `ProcessRefund` for a captured payment, `RestockOrder` (new `inventory.v1` RPC, idempotent by order ID, writes `RETURN` movements) for confirmed stock on cancellation.	Must
SAGA-008	Reconciler every 30 s: any order in a non-terminal state with `updated_at` older than its step SLA (RESERVE 30 s, PAYMENT 60 s, CONFIRM 30 s, COMPENSATING 120 s) is re-driven; after 15 min it moves to `REQUIRES_ATTENTION` and emits `order.requires_attention`.	Must

ID	Requirement	Priority
SAGA-009	If `ConfirmStock` fails because the reservation expired while payment was slow, the saga refunds the payment and ends `FAILED` with `failure_code=RESERVATION_EXPIRED`.	Must
SAGA-010	`inventory.stock.released{reason: EXPIRED_TTL}` received for an order still in `RESERVED`/`PAYMENT_PENDING` moves it to `FAILED` (`RESERVATION_EXPIRED`) after confirming payment did not succeed.	Must
SAGA-011	Payment retry by the customer is a new order in MVP (no partial re-entry).	Should

11.2 Order state machine

```
stateDiagram-v2
  [*] --> PENDING: CreateOrder (202)
  PENDING --> RESERVED: ReserveStock OK
  PENDING --> FAILED: INSUFFICIENT_STOCK / product inactive
  RESERVED --> PAYMENT_PENDING: ProcessPayment sent
  PAYMENT_PENDING --> PAID: payment SUCCEEDED
  PAYMENT_PENDING --> COMPENSATING: payment FAILED (declined)
  PAYMENT_PENDING --> PAYMENT_UNKNOWN: timeout / UNAVAILABLE
  PAYMENT_UNKNOWN --> PAID: status query = SUCCEEDED
  PAYMENT_UNKNOWN --> COMPENSATING: status query = FAILED / NOT_FOUND after retry
budget
  PAYMENT_UNKNOWN --> REQUIRES_ATTENTION: unresolved after 15 min
  PAID --> CONFIRMED: ConfirmStock OK
  PAID --> COMPENSATING: reservation expired (refund)
  COMPENSATING --> FAILED: stock released and/or refund done
  COMPENSATING --> REQUIRES_ATTENTION: compensation stuck 15 min
  RESERVED --> CANCELLATION_REQUESTED: cancel
  PAYMENT_PENDING --> CANCELLATION_REQUESTED: cancel (waits for payment outcome)
  PAID --> CANCELLATION_REQUESTED: cancel
  CONFIRMED --> CANCELLATION_REQUESTED: cancel (pre-fulfilment)
  CANCELLATION_REQUESTED --> CANCELLED: release/restock + refund done
  CONFIRMED --> FULFILLED: ops marks fulfilled
  FAILED --> [*]
  CANCELLED --> [*]
  FULFILLED --> [*]
```

Terminal states: `CONFIRMED` is stable until fulfilment or cancellation; `FAILED`, `CANCELLED`, `FULFILLED` are final. `REQUIRES_ATTENTION` is resolved by an admin action (`retry-saga` or `force-fail` with mandatory reason, both audited).

11.3 Successful order lifecycle (Workflow A)

```
sequenceDiagram
  autonumber
  participant C as Client
```

```
  participant GW as Gateway
  participant ID as Identity
  participant ORD as Order
  participant CAT as Catalog
  participant INV as Inventory
  participant PAY as Payment
  participant MQ as RabbitMQ
  participant NOT as Notification
  C->>GW: POST /orders (Idempotency-Key K, JWT)
  GW->>ID: ValidateSession(sid)
  GW->>ORD: CreateOrder(K, items) [deadline 3s]
  ORD->>CAT: GetProducts(ids, STRONG) [800ms]
  ORD->>ORD: Tx: INSERT order PENDING, items (price snapshot), saga_state, outbox
order.created
  ORD-->>GW: orderId, PENDING
  GW-->>C: 202 Accepted
  ORD->>INV: ReserveStock(orderId, lines) [2s]
  INV->>INV: Tx: lock rows, reserved += q, outbox inventory.stock.reserved
  INV-->>ORD: reservationId, expiresAt
  ORD->>ORD: Tx: RESERVED -> PAYMENT_PENDING
  ORD->>PAY: ProcessPayment(key=orderId:1, amount) [3s]
  PAY->>PAY: Tx: payment SUCCEEDED, outbox payment.succeeded
  PAY-->>ORD: SUCCEEDED, paymentId
  ORD->>ORD: Tx: PAID
  ORD->>INV: ConfirmStock(reservationId) [2s]
  INV->>INV: Tx: on_hand -= q, reserved -= q, SALE movements, outbox
inventory.stock.confirmed
  INV-->>ORD: CONFIRMED
  ORD->>ORD: Tx: CONFIRMED, history, outbox order.confirmed
  ORD--)MQ: relay publishes order.confirmed
  MQ--)NOT: deliver
  NOT->>NOT: dedupe, INSERT notification for customer
  C->>GW: GET /orders/{id}/status -> CONFIRMED
```

11.4 Payment failure and compensation (Workflow B)

```
sequenceDiagram
  autonumber
  participant ORD as Order saga
  participant INV as Inventory
  participant PAY as Payment
  participant REC as Order reconciler
  participant MQ as RabbitMQ
  participant NOT as Notification
  ORD->>PAY: ProcessPayment(key=orderId:1)
  alt declined
    PAY->>PAY: Tx: payment FAILED, outbox payment.failed
    PAY-->>ORD: FAILED (decline_code)
    ORD->>ORD: Tx: PAYMENT_PENDING -> COMPENSATING
    ORD->>INV: ReleaseStock(reservationId, PAYMENT_FAILED)
    alt release OK (or already released)
      INV-->>ORD: RELEASED
      ORD->>ORD: Tx: FAILED, outbox order.failed
      ORD--)MQ: order.failed
      MQ--)NOT: notify customer
```

```
    else Inventory unavailable
      INV--xORD: UNAVAILABLE
      ORD->>ORD: stay COMPENSATING, next_check_at = now()+backoff
      REC->>ORD: re-drive after SLA (120s)
      ORD->>INV: ReleaseStock (same reservationId, idempotent)
      Note over INV: even if never retried, the reservation TTL sweeper releases
stock after 15 min
    end
  else timeout after Payment actually captured
    PAY--xORD: DEADLINE_EXCEEDED
    ORD->>ORD: Tx: PAYMENT_UNKNOWN
    ORD->>PAY: ProcessPayment(same key) -> replayed SUCCEEDED
    ORD->>ORD: Tx: PAID, continue to ConfirmStock
  end
```

11.5 Cancellation

State at cancel	Actions (in order)	End state
`PENDING`	mark cancelled; if a reservation appears later, the saga sees `CANCELLATION_REQUESTED` and releases it	`CANCELLED`
`RESERVED`	`ReleaseStock(CANCELLED)`	`CANCELLED`
`PAYMENT_PENDING` / `PAYMENT_UNKNOWN`	wait for payment outcome; if succeeded → refund; then release	`CANCELLED`
`PAID`	`ProcessRefund(full)` → `ReleaseStock`	`CANCELLED`
`CONFIRMED`	`ProcessRefund(full)` → `RestockOrder`	`CANCELLED`
`FULFILLED`, `FAILED`, `CANCELLED`	reject `409 INVALID_STATE_TRANSITION` (or 200 replay for an already-cancelled order with same idempotency key)	unchanged

Refund failure (simulated) keeps the order in `CANCELLATION_REQUESTED`; the reconciler retries with the same `refund_key` and escalates after 15 min.

12. Security and privacy requirements

Security is enforced in depth — gateway, service, database and pipeline — and mapped to the OWASP API Security Top 10 (2023); no legal compliance or certification is claimed.

12.1 Requirements

ID	Requirement	Priority
SEC-001	Authentication per Section 9.4: ES256 JWT, rotating refresh tokens, reuse detection, server-side session validation.	Must

ID	Requirement	Priority
SEC-002	Authorization enforced in the owning service for every RPC (permission + ownership + warehouse scope); gateway checks are additive.	Must
SEC-003	Object-level authorization: resource lookups include the caller's scope in the query; cross-tenant access returns 404, not 403.	Must
SEC-004	Property-level authorization: request schemas are `.strict()` (no mass assignment); response DTOs are explicit allow-lists (e.g. never `password_hash`, `token_hash`).	Must
SEC-005	Input validation with Zod at the gateway and again in services; size, range, enum and format limits on every field; reject unknown fields.	Must
SEC-006	SQL injection prevention: parameterized queries only; dynamic sort/filter columns mapped from allow-lists, never interpolated.	Must
SEC-007	Rate limiting and resource limits per API-009/API-010; pagination max 100; batch RPCs ≤ 100 items.	Must
SEC-008	Security headers via `helmet`: HSTS (prod), `X-Content-Type-Options`, `Referrer-Policy: no-referrer`, CSP for the SPA (`default-src 'self'`), `frame-ancestors 'none'`.	Must
SEC-009	CORS allow-list of exact origins from config, credentials only for the dashboard origin; no wildcard with credentials.	Must
SEC-010	CSRF: refresh/logout endpoints use `SameSite=Strict` cookie + required custom header `X-Requested-With: stockflow`; all other endpoints use bearer tokens (not cookies).	Must
SEC-011	Secrets (DB passwords, JWT private keys, mTLS keys, MinIO keys) supplied via Docker secrets / env files excluded by `.gitignore`; `.env.example` contains placeholders only; production uses a secret manager. Gitleaks blocks commits and CI.	Must
SEC-012	Service-to-service mTLS with per-service certificates and per-RPC caller allow-lists (GRPC-005).	Must
SEC-013	Private object storage: bucket policy denies public access; uploads via presigned PUT (5 min, exact content-type and length conditions); downloads via presigned GET (5 min) after an authorization check; object keys are random UUIDs (no user-supplied filenames); magic-byte verification on finalize; path traversal impossible by construction.	Should

ID	Requirement	Priority
SEC-014	Sensitive-data minimisation: no card data; PII limited to email, name, shipping address; events carry IDs only; logs redact `authorization`, `cookie`, `password`, `*token*`, `email`, `address` via pino redaction paths.	Must
SEC-015	Safe errors: `INTERNAL` maps to a generic 500 with correlation ID only; stack traces logged server-side, never returned. Express `x-powered-by` disabled.	Must
SEC-016	Account protection: Argon2id (m=19 MiB, t=2, p=1 minimum), progressive lockout per FR-008, uniform responses and timing for unknown email vs wrong password.	Must
SEC-017	Audit logging of all security events (FR-045) to append-only storage; audit records contain no secrets.	Must
SEC-018	Dependency management: lockfile committed, `pnpm audit --prod` and Trivy fs scan in CI, Renovate/Dependabot weekly; critical CVEs block merge, high CVEs must be triaged within 7 days.	Must
SEC-019	Container hardening: non-root UID, read-only root filesystem, `cap_drop: [ALL]`, `no-new-privileges`, distroless or Alpine runtime, no shell in production images where possible.	Must
SEC-020	Internal ports (gRPC, PostgreSQL, Redis, RabbitMQ AMQP) are not published to the host in the default Compose profile; only gateway 8080, RabbitMQ UI 15672, Grafana 3000, MinIO console 9001 bind to `127.0.0.1`.	Must
SEC-021	Unsafe patterns forbidden (Semgrep-enforced): `eval`, `new Function`, `child_process.exec` with variable input, `jwt.decode` without verify, `Math.random` for tokens, disabled TLS verification, `JSON.parse` of untrusted input without schema validation.	Must
SEC-022	Data retention per DB-011; user deletion request anonymises PII in identity and order snapshots while keeping ledger integrity (process defined, implementation Could).	Should
SEC-023	Threat model (STRIDE) for the order and auth flows reviewed before Phase 4 exit.	Should

12.2 OWASP API Security Top 10 (2023) mapping

Risk	Primary controls
API1 Broken Object Level Authorization	SEC-002, SEC-003, FR-AUTHZ-02; E2E tests for cross-customer access
API2 Broken Authentication	SEC-001, SEC-016, REDIS-014–018
API3 Broken Object Property Level Authorization	SEC-004
API4 Unrestricted Resource Consumption	SEC-007, API-009, gRPC max message size, DB statement timeout
API5 Broken Function Level Authorization	FR-AUTHZ-03 deny-by-default route registry, permission matrix tests
API6 Unrestricted Access to Sensitive Business Flows	Idempotency keys, order rate limit (10 orders/min/user), refund limits
API7 Server Side Request Forgery	No user-supplied URLs fetched server-side; presigned URLs generated, never fetched from client input
API8 Security Misconfiguration	SEC-008, SEC-009, SEC-019, SEC-020, Semgrep + Trivy config scanning
API9 Improper Inventory Management	OpenAPI committed and diffed (API-002), versioned paths, no undocumented routes (route registry lint)
API10 Unsafe Consumption of APIs	Internal gRPC responses and events schema-validated (GRPC-003, MQ-010)

Compliance note: the system is designed with data-protection principles (minimisation, retention, access control) in mind, but no claim of GDPR, DPDP Act 2023, PCI DSS or SOC 2 compliance is made. Applicable obligations depend on jurisdiction and deployment and require stakeholder and legal confirmation (OQ-07).

13. Docker and local development

One command (`make up`) builds hardened multi-stage images and starts the full platform — 8 services, their workers, PostgreSQL, Redis, RabbitMQ, MinIO and the observability stack — in health-gated order on an 8 GB laptop.

13.1 Requirements

ID	Requirement	Priority
DOCKER-001	One Dockerfile per deployable service, multi-stage: `deps` (pnpm fetch, cached), `build` (tsc, prune to prod deps via `pnpm deploy`), `runtime`.	Must
DOCKER-002	Runtime image `gcr.io/distroless/nodejs24-debian12:nonroot` (or `node:24-alpine` with `USER node`); runs as non-root UID 65532/1000; final image < 200 MB.	Must
DOCKER-003	`.dockerignore` excludes `node_modules`, `.git`, `.env*`, `coverage`, `test`, `*.pem`, `docs`.	Must
DOCKER-004	All configuration through environment variables validated by a Zod config schema at startup; invalid config → exit code 78 with a clear message. Secrets via `*_FILE` env pointing at Docker secrets.	Must
DOCKER-005	`HEALTHCHECK` defined (Compose-level `healthcheck` for distroless using a bundled `node healthcheck.js`); `depends_on: condition: service_healthy` for infra and `service_completed_successfully` for migrations.	Must
DOCKER-006	Graceful shutdown on SIGTERM: stop accepting HTTP/gRPC, cancel consumers, finish in-flight work up to 20 s, commit/ack, close pools; `stop_grace_period: 30s`; `init: true` (tini) for signal forwarding.	Must
DOCKER-007	Named volumes for `pgdata`, `redisdata`, `rabbitdata`, `miniodata`; `make reset` removes them.	Must
DOCKER-008	Migration and seed run as one-shot containers (`migrate-<svc>`, `seed`) using the same image with a different command.	Must
DOCKER-009	Resource defaults: services 256 MB / 0.5 CPU, workers 192 MB / 0.25 CPU, PostgreSQL 1 GB, RabbitMQ 512 MB, Redis 256 MB; total < 6 GB.	Should
DOCKER-010	Compose profiles: default (app + infra), `observability` (OTel collector, Prometheus, Grafana, Jaeger), `tools` (pgAdmin, RedisInsight), `test` (ephemeral infra for CI).	Should
DOCKER-011	Images labelled with OCI labels (`org.opencontainers.image.revision`, `version`, `source`) and tagged `<svc>:<semver>-<gitsha>`; never `latest` in CI.	Must

ID	Requirement	Priority
DOCKER-012	No secrets in image layers (verified by Trivy secret scan and `docker history` check in CI).	Must

13.2 Reference Dockerfile (order service)

```
# syntax=docker/dockerfile:1.7
FROM node:24-alpine AS base
RUN corepack enable
WORKDIR /repo

FROM base AS deps
COPY pnpm-lock.yaml pnpm-workspace.yaml ./
RUN --mount=type=cache,id=pnpm,target=/root/.local/share/pnpm/store pnpm fetch
--frozen-lockfile

FROM deps AS build
COPY . .
RUN pnpm install --offline --frozen-lockfile \
 && pnpm --filter @stockflow/order... build \
 && pnpm --filter @stockflow/order deploy --prod /out

FROM gcr.io/distroless/nodejs24-debian12:nonroot AS runtime
ENV NODE_ENV=production
WORKDIR /app
COPY --from=build --chown=nonroot:nonroot /out ./
USER nonroot
EXPOSE 50051 9464
CMD ["dist/main.js"]          # workers: ["dist/outbox-relay.js"],
["dist/saga-worker.js"]
```

13.3 Compose excerpt

```
services:
  postgres:
    image: postgres:17-alpine
    environment: { POSTGRES_PASSWORD_FILE: /run/secrets/pg_root }
    volumes: [pgdata:/var/lib/postgresql/data,
./postgres/init:/docker-entrypoint-initdb.d:ro]  # creates 7 DBs + roles
    healthcheck: { test: ["CMD-SHELL", "pg_isready -U postgres"], interval: 5s,
retries: 20 }
    secrets: [pg_root]
  redis:
    image: redis:7.4-alpine
    command: ["redis-server", "/usr/local/etc/redis/redis.conf"]   #
notify-keyspace-events Ex, ACL users
    healthcheck: { test: ["CMD", "redis-cli", "ping"], interval: 5s }
  rabbitmq:
    image: rabbitmq:4.1-management-alpine
    volumes: [rabbitdata:/var/lib/rabbitmq,
./rabbitmq/definitions.json:/etc/rabbitmq/definitions.json:ro]
    ports: ["127.0.0.1:15672:15672"]
    healthcheck: { test: ["CMD", "rabbitmq-diagnostics", "-q",
```

```
"check_port_connectivity"], interval: 10s }
  migrate-order:
    image: stockflow/order:${TAG}
    command: ["dist/migrate.js"]
    depends_on: { postgres: { condition: service_healthy } }
  order:
    image: stockflow/order:${TAG}
    env_file: [.env]                      # git-ignored; copy from .env.example
    secrets: [order_db_password, mtls_order_key]
    read_only: true
    cap_drop: [ALL]
    security_opt: ["no-new-privileges:true"]
    init: true
    stop_grace_period: 30s
    deploy: { resources: { limits: { memory: 256M, cpus: "0.5" } } }
    depends_on:
      migrate-order: { condition: service_completed_successfully }
      rabbitmq: { condition: service_healthy }
      redis: { condition: service_healthy }
    healthcheck: { test: ["CMD", "/nodejs/bin/node", "dist/healthcheck.js"],
interval: 10s, start_period: 20s }
  order-outbox-relay:  { image: stockflow/order:${TAG}, command:
["dist/outbox-relay.js"], depends_on: { order: { condition: service_healthy } } }
  order-saga-worker:   { image: stockflow/order:${TAG}, command:
["dist/saga-worker.js"],  depends_on: { order: { condition: service_healthy } } }
  api-gateway:
    image: stockflow/api-gateway:${TAG}
    ports: ["127.0.0.1:8080:8080"]        # only public entry point
```

13.4 Deployment view

```
flowchart TB
  subgraph host[Developer host - 127.0.0.1 only]
    P8080[:8080 gateway]
    P15672[:15672 RabbitMQ UI]
    P3000[:3000 Grafana]
    P9001[:9001 MinIO console]
  end
  subgraph net_edge[network: edge]
    GW[api-gateway]
    UI[dashboard - nginx static]
  end
  subgraph net_app[network: app - internal]
    ID[identity] ; CAT[catalog] ; INV[inventory] ; ORD[order] ; PAY[payment] ;
NOT[notification] ; AUD[audit]
    W1[outbox relays x5] ; W2[saga worker] ; W3[reconcilers x3] ; W4[consumers]
  end
  subgraph net_data[network: data - internal]
    PG[(postgres:17)] ; RD[(redis:7.4)] ; MQ[(rabbitmq:4.1)] ; S3[(minio)]
  end
  subgraph obs[profile: observability]
    OTEL[otel-collector] --> PROM[prometheus] ; OTEL --> JAE[jaeger] ; PROM -->
GRAF[grafana]
  end
  JOBS[[one-shot: migrate-* , seed]] --> PG
```

```
P8080 --> GW
GW --> ID & CAT & INV & ORD & PAY & NOT & AUD
ID & CAT & INV & ORD & PAY & NOT & AUD --> PG
ORD & INV & PAY & ID & CAT --> MQ
ID & CAT & GW --> RD
CAT --> S3
```

13.5 Developer commands

Task	Command
First-time setup (certs, `.env` from example, secrets)	`make bootstrap`
Start everything (build + up, waits for healthy)	`make up` (= `docker compose up -d --build --wait`)
Start with observability	`make up PROFILES=observability`
Run migrations only	`make migrate` (= `docker compose run --rm migrate-<svc>` for each)
Seed sample data (3 warehouses, 40 products, demo users per role)	`make seed`
Unit tests (all / one service)	`pnpm test` / `pnpm --filter @stockflow/inventory test`
Integration tests (Testcontainers)	`pnpm test:integration`
E2E against running Compose stack	`make e2e`
Coverage report	`pnpm test:coverage`
Logs	`docker compose logs -f order order-saga-worker`
Inspect queues	`docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged` or UI at `http://127.0.0.1:15672`
Replay DLQ	`pnpm mq:replay --queue notification.dlq --dry-run`
Semgrep	`make semgrep` (= `semgrep scan --config .semgrep/ --config p/typescript --config p/nodejs --config p/jwt --config p/secrets --error`)
Chaos: stop a dependency	`docker compose stop payment` / `docker compose pause rabbitmq`

Task	Command
Shut down (keep data) / reset	`make down` / `make reset` (removes volumes)

14. Semgrep and CI security gates

Every pull request must pass formatting, lint, type-check, unit, integration, contract, coverage, Semgrep (zero ERROR findings), Buf breaking-change, Gitleaks and Trivy gates before merge; a clean scan is evidence, not proof, of security.

14.1 Semgrep requirements

ID	Requirement	Priority
SAST-001	`.semgrep.yml` + `.semgrep/rules/*.yml` committed; Semgrep CLI version pinned in CI (`semgrep/semgrep:<pinned>` image) and in `.tool-versions`.	Must
SAST-002	Rule selection: registry packs `p/typescript`, `p/nodejs`, `p/expressjs`, `p/jwt`, `p/secrets`, `p/owasp-top-ten`, `p/sql-injection`, `p/dockerfile`, `p/docker-compose` + project rules below. Packs are reviewed quarterly; rule availability and behaviour depend on the installed Semgrep version and registry state, so packs are snapshot-tested on upgrade.	Must
SAST-003	Severity gates: `ERROR` blocks merge; `WARNING` blocks merge for files under `services/*/src/**/auth*`, `**/payment*`, `**/*repository*` and is otherwise reported as a PR comment; `INFO` is report-only.	Must
SAST-004	Output SARIF uploaded to GitHub code scanning (or GitLab SAST report) and JSON artifact retained 90 days.	Must
SAST-005	Diff-aware scan on PRs (`--baseline-commit origin/main`) plus a full scan nightly on `main`.	Must
SAST-006	Suppressions only via inline `// nosemgrep: <rule-id> -- <justification, ticket>`; a custom rule fails any `nosemgrep` without a rule ID and justification; suppressions reviewed by a CODEOWNER from `@security`.	Must
SAST-007	Every custom rule has a test file (`.semgrep/rules/<rule>.ts` with `// ruleid:` and `// ok:` annotations) run by `semgrep --test` in CI.	Must
SAST-008	Findings triage SLA: ERROR fixed before merge; WARNING on `main` fixed or justified within 14 days; false positives reported upstream or rule tuned.	Should

Project-specific rules (all `ERROR` unless noted):

Rule ID	Detects
`sf-sql-string-concat`	Template literals or `+` building strings passed to `pool.query`, `client.query`, `sql.raw`
`sf-jwt-decode-without-verify`	`jose.decodeJwt` / `jsonwebtoken.decode` result used for authorization
`sf-jwt-alg-allowlist`	`jwtVerify` called without `algorithms` option
`sf-direct-amqp-publish`	`channel.publish` / `sendToQueue` outside `packages/mq-kit` and `outbox-relay` (MQ-001)
`sf-grpc-call-without-deadline`	Generated client method invoked without `{ deadline }` call options (GRPC-002)
`sf-express-route-without-permission`	`router.<verb>(` without the `requirePermission(` middleware in the chain (FR-AUTHZ-03)
`sf-insecure-random-token`	`Math.random()` in files under `auth/`, `token`, `idempotency`
`sf-log-sensitive` (WARNING)	`logger.*(` with objects containing `password`, `token`, `authorization`, `cookie` keys
`sf-tls-reject-unauthorized`	`rejectUnauthorized: false`, `NODE_TLS_REJECT_UNAUTHORIZED`, `grpc.credentials.createInsecure()` outside `test/`
`sf-unsafe-path-join`	`path.join/resolve` with request-derived input in file or object-key handling
`sf-child-process`	`child_process.exec/execSync` anywhere in `services/`
`sf-json-parse-untrusted` (WARNING)	`JSON.parse` on message/request content not followed by a Zod `.parse`
`sf-nosemgrep-justification`	`nosemgrep` without rule id + reason

Example custom rule:

```
rules:
  - id: sf-sql-string-concat
    languages: [typescript]
    severity: ERROR
    message: "SQL built from string interpolation. Use parameters ($1) or Kysely."
    metadata: { cwe: "CWE-89", owasp: "A03:2021", category: security }
    patterns:
      - pattern-either:
          - pattern: $DB.query(`...${$X}...`, ...)
```

```
          - pattern: $DB.query("..." + $X + "...", ...)
          - pattern: sql.raw(`...${$X}...`)
    paths:
      include: ["services/*/src/**", "packages/*/src/**"]
      exclude: ["**/*.test.ts", "services/*/migrations/**"]   # migrations reviewed
separately
```

Local commands: `make semgrep` (full), `semgrep scan --config auto .` (exploratory, not the gate), `semgrep --test .semgrep/rules`.

14.2 Complementary controls

Semgrep does not replace: code review with a security checklist for auth/payment/inventory changes (CODEOWNERS), the authorization and failure-path test suites, Trivy image + dependency scanning, Gitleaks secret scanning, ESLint `eslint-plugin-security`, Buf breaking checks, and runtime checks (rate limits, mTLS, audit). OWASP ZAP baseline against the gateway is a Could (Phase 5).

14.3 CI pipeline

```
flowchart LR
  A[checkout + pnpm install --frozen-lockfile] --> B[format check + ESLint]
  A --> C[tsc --noEmit all packages]
  A --> D[buf lint + buf breaking]
  A --> G[Gitleaks]
  A --> S[Semgrep diff-aware + semgrep --test]
  B & C & D --> U[unit tests + coverage per service - matrix]
  U --> I[integration tests - Testcontainers PG/Redis/RabbitMQ/MinIO]
  I --> K[contract tests - gRPC + event schemas]
  K --> COV{coverage thresholds}
  COV --> BLD[docker buildx per service - matrix, cache]
  BLD --> T[Trivy image scan: vuln + secret + misconfig]
  T --> E[E2E on Compose stack incl. failure scenarios]
  S --> GATE{all required checks green?}
  G --> GATE
  E --> GATE
  GATE -- no --> X[block merge, annotate PR, SARIF upload]
  GATE -- yes, main branch --> PUB[push images tag semver-sha + SBOM + cosign sign]
```

Stage	Blocks merge	Threshold
Lockfile / install	Yes	`--frozen-lockfile` succeeds
Prettier + ESLint	Yes	0 errors (warnings allowed ≤ existing baseline)
TypeScript	Yes	0 errors
Buf lint / breaking	Yes	0 violations vs `main` (breaking allowed only with `v2` package)
Unit tests	Yes	100% pass
Integration + contract tests	Yes	100% pass; flaky test quarantine requires ticket
Coverage	Yes	Section 15.5 thresholds

Stage	Blocks merge	Threshold
Semgrep	Yes	0 ERROR; WARNING per SAST-003
Gitleaks	Yes	0 findings
Docker build	Yes	All images build; size budget ≤ 200 MB
Trivy	Yes	0 CRITICAL with fix available; HIGH triaged (issue opened, 7-day SLA)
E2E	Yes on `main` merge queue; nightly full chaos suite	100% pass
Publish	—	Only from `main` after all gates; SBOM (Syft) attached

Target pipeline duration ≤ 15 min for PRs (parallel matrix, Turborepo remote cache, Docker layer cache).

15. Testing: unit, integration, contract and end-to-end

Tests prove invariants and failure paths, not coverage numbers: pure domain logic is unit-tested with mocks, but every claim about PostgreSQL, Redis, RabbitMQ or gRPC behaviour is proven against real disposable instances.

15.1 Test pyramid and tooling

Layer	Scope	Tooling	Runs	Target count (MVP)
Unit	`domain/` and `application/` with ports mocked	Vitest, `fast-check` for property tests	every PR, < 60 s	~600
Integration	One service + real PG/Redis/RabbitMQ/MinIO	Vitest + Testcontainers, Supertest for gateway	every PR, < 6 min	~150
Contract	gRPC provider vs consumer fixtures; event producer fixtures vs schemas	Vitest + generated clients, Zod/AsyncAPI	every PR	~60
E2E	Full Compose stack via gateway REST	Vitest + Supertest + `toxiproxy` / `docker compose pause`	merge queue + nightly	~35 scenarios
Load / concurrency	Order creation and reservation	k6	nightly / pre-release	3 profiles

15.2 Unit tests (TEST-001 – TEST-014, all Must)

ID	Subject	Key cases
TEST-001	Inventory allocation & invariants	priority ordering; split across warehouses; insufficient total → error; property test: for any sequence of reserve/confirm/release/adjust, `available ≥ 0` and ledger sum = on_hand
TEST-002	Order state machine	every legal transition; every illegal pair rejected (exhaustive table-driven test)
TEST-003	Price & quantity validation	qty bounds 1–1000, 50 lines, totals in minor units, overflow guard (`bigint`)
TEST-004	Permission checks	matrix in 4.2 encoded as test table; ownership and warehouse scope
TEST-005	JWT validation	expired, `nbf` future, wrong `aud`/`iss`, `alg: none`, HS256 with public key, unknown `kid`, clock skew ± 30 s
TEST-006	Refresh rotation & reuse	rotation, reuse outside grace → family revoked, reuse inside grace → same successor
TEST-007	Session lifecycle	idle vs absolute expiry, cap of 10, revoke clears shadow key
TEST-008	Cache-aside	hit, miss+fill, tombstone, invalidation after write, Redis error → DB fallback
TEST-009	Idempotency	same key + same hash → replay; same key + different hash → conflict; concurrent first use
TEST-010	Retry policy	retryable status set, backoff bounds with jitter, deadline budget respected, never retries non-idempotent
TEST-011	Event schema validation	valid v1, unknown version, missing field, extra field tolerance
TEST-012	Outbox relay decisions	ack → published, nack → backoff, timeout → retry, returned (unroutable) → failure
TEST-013	Consumer dedup & error classification	duplicate → ack no-op; transient → retry tier; permanent → DLQ; retry count header
TEST-014	Compensation & reconciliation decisions	per-state compensation plan (11.5); unknown outcome → no compensation; SLA breach → re-drive / REQUIRES_ATTENTION

Mocks are allowed only at port boundaries (repositories, gRPC clients, publishers, clock); a mocked repository test never counts as evidence that SQL works.

15.3 Integration tests (Must)

ID	Proves
TEST-015	PostgreSQL constraints: `available_non_negative`, unique SKU (case-insensitive), `refund_le_amount`, append-only movements (UPDATE denied for app role)
TEST-016	50 parallel `ReserveStock` calls for 1 unit each against `on_hand = 10` → exactly 10 succeed, `reserved = 10`; no deadlocks across multi-SKU orders in random order
TEST-017	Redis TTL & keyspace notification: shadow key expires → handler marks session EXPIRED once with 3 replicas subscribed
TEST-018	Session reconciler finds sessions whose notification was dropped (subscriber disconnected during expiry)
TEST-019	RabbitMQ: publisher confirms; unroutable message returned; consumer ack after commit; crash before ack → redelivery → dedup
TEST-020	Retry tiers and DLQ: transient failure 10× → message lands in DLQ with `x-death`; permanent failure → DLQ immediately; replay CLI restores it
TEST-021	Outbox: broker stopped → business writes succeed, backlog grows; broker started → backlog drains, each event delivered ≥ 1 time
TEST-022	gRPC: real server + client over mTLS; deadline propagation; status mapping table (7.4) at the gateway; unauthenticated client cert rejected
TEST-023	MinIO: presigned PUT with wrong content-type rejected; unauthorized GET denied; finalize rejects mismatched magic bytes
TEST-024	Gateway (Supertest): error schema, correlation ID echo, rate limiting 429, body size 413, unknown query params 400

Testcontainers instances use the same images and config (`redis.conf`, `definitions.json`, init SQL) as Compose to avoid drift.

15.4 End-to-end scenarios (Must unless noted)

ID	Scenario	Expected
TEST-025	Successful order (`pm_ok`)	`CONFIRMED` ≤ 5 s; on_hand decremented; 1 payment; notification created

ID	Scenario	Expected
TEST-026	Duplicate submission (same key × 5, concurrently)	1 order, 1 reservation, 1 payment; all responses identical
TEST-027	Concurrent last-stock: 200 orders × 1 unit, stock 50	50 `CONFIRMED`, 150 `FAILED(INSUFFICIENT_STOCK)`, 0 negative balances
TEST-028	Payment declined (`pm_declined`)	`FAILED`, reservation released, `available` restored, customer notified
TEST-029	Timeout after success (`pm_slow_success`: captures then sleeps past deadline)	Order `CONFIRMED`, exactly 1 `SUCCEEDED` payment row, 0 refunds
TEST-030	Payment service down during saga	Order reaches `PAYMENT_UNKNOWN` then resolves after restart; or reservation expires and order `FAILED`
TEST-031	Cancel at each state (`RESERVED`, `PAID`, `CONFIRMED`)	End `CANCELLED`, refunds equal captured, stock restored via release/restock
TEST-032	Session revocation	Revoke from second device → first device's next request 401 within 1 s
TEST-033	Refresh token reuse	Replay old token → 401, all tokens in family dead, security notification + audit event
TEST-034	Session expiry with Redis restart mid-TTL	Audit shows `identity.session.expired` (detectedBy=reconciler) within 6 min
TEST-035	RabbitMQ paused 5 min during order traffic	No 5xx on order creation; backlog drains; notifications appear once each
TEST-036	Inventory worker killed mid-consume	Redelivered message processed once (no duplicate notification/audit)
TEST-037	Low stock	Adjustment crossing threshold → exactly 1 alert per crossing
TEST-038	Cross-customer access	Customer B `GET /orders/{A's id}` → 404; warehouse operator outside assignment → 404
TEST-039	Redis down	Product reads succeed (slower), login still works via PG, rate limiting falls back (Should)

15.5 Coverage and quality gates

- **TEST-040 (Must)** Thresholds enforced per service by Vitest (`coverage.thresholds`): `domain/` and `application/` ≥ 85% lines and ≥ 80% branches; security-critical modules (`identity/src/domain/token*`, `session*`, `authz`, `inventory/src/domain/reservation*`,

`order/src/domain/saga*`, `payment/src/domain/*`) ≥ 95% lines and ≥ 90% branches; overall ≥ 80% lines/branches.

- **TEST-041 (Must)** Exclusions limited to generated code (`packages/contracts/gen/**`), `main.ts` bootstraps and migrations; any new exclusion requires CODEOWNER review. Coverage is measured on unit + integration combined.
- **TEST-042 (Must)** Every Must acceptance criterion (Section 19) is tagged in at least one test name (`[AC-010]`); CI script fails if an AC tag is missing.
- **TEST-043 (Should)** Mutation testing (Stryker) on inventory and saga domain modules nightly, mutation score ≥ 70%.
- **TEST-044 (Must)** Tests are deterministic: injected clock, seeded UUIDs/randomness, no `sleep` without polling helper (`eventually()` with timeout).

16. Observability and operational requirements

Every request is traceable from the browser to the last consumer by one correlation ID and one OpenTelemetry trace; liveness never depends on downstreams, so a dependency blip degrades readiness instead of triggering restart loops.

16.1 Requirements

ID	Requirement	Priority
OBS-001	`/livez` returns 200 if the event loop responds (lag < 1 s) and the process is not shutting down; it checks **no** external dependency.	Must
OBS-002	`/readyz` checks hard dependencies with 500 ms timeouts and returns 503 with a per-dependency JSON breakdown when any fails; during shutdown it returns 503 first (drain). gRPC health mirrors it.	Must
OBS-003	Structured JSON logs (pino) with fields `ts`, `level`, `service`, `version`, `correlationId`, `traceId`, `spanId`, `userId?`, `msg`, `err{type,message,stack}`; no `console.log` (lint).	Must
OBS-004	Correlation ID propagation: HTTP header → gRPC metadata → outbox headers → AMQP headers → consumer logs; W3C `traceparent` propagated alongside.	Must
OBS-005	OpenTelemetry auto-instrumentation for http, express, grpc, pg, ioredis, amqplib; spans for saga steps and outbox publish; consumer spans linked to producer spans.	Must
OBS-006	Prometheus metrics on `:9464/metrics` (internal network only).	Must
OBS-007	Grafana dashboards committed as JSON: API overview, gRPC, saga, RabbitMQ, outbox, Redis, PostgreSQL, sessions.	Should
OBS-008	Alert rules (Prometheus) committed for the conditions in 16.3.	Should

ID	Requirement	Priority
OBS-009	Runbooks in `docs/runbooks/` for each alert and failure mode in 16.4.	Must
OBS-010	Graceful shutdown order: readiness 503 → wait 5 s → stop HTTP/gRPC accept → cancel AMQP consumers → drain in-flight (≤ 20 s) → flush OTel → close pools → exit 0.	Must

16.2 Hard vs soft dependencies (readiness)

Service	Hard (fail readiness)	Soft (degrade, log, metric)
api-gateway	— (downstream failures are per-route 503s)	Redis (rate limit fallback)
identity	PostgreSQL	Redis (PG fallback), RabbitMQ (outbox buffers)
catalog	PostgreSQL	Redis, RabbitMQ, MinIO (media endpoints 503)
inventory	PostgreSQL	RabbitMQ, Redis
order	PostgreSQL	RabbitMQ; downstream gRPC (saga waits)
payment	PostgreSQL	RabbitMQ
notification / audit	PostgreSQL, RabbitMQ	—
outbox-relay	PostgreSQL	RabbitMQ (relay retries; readiness stays up, backlog alert fires)

16.3 Key metrics and alerts

Metric	Labels	Alert (example)
`http_server_request_duration_seconds` (histogram)	route, method, status	p95 > 500 ms for 10 min; 5xx ratio > 2% for 5 min
`grpc_client_handling_seconds`, `grpc_client_status_total`	service, method, code	`UNAVAILABLE+DEADLINE_EXCEEDED` > 5% for 5 min
`saga_orders_in_state` (gauge), `saga_step_duration_seconds`	state, step	`REQUIRES_ATTENTION` > 0; `COMPENSATING` older than 2 min
`outbox_backlog`, `outbox_oldest_unpublished_seconds`	service	oldest > 60 s for 5 min
`rabbitmq_queue_messages_ready` (plugin), `mq_dlq_total`	queue	DLQ depth > 0 for 5 min; ready > 1000 for 10 min

Metric	Labels	Alert (example)
`mq_consume_total{result}`	consumer, result=ok/dup/retry/dlq	dup ratio > 20% (redelivery storm)
`cache_requests_total{result}`	cache	hit rate < 60% for 30 min
`redis_errors_total`, `redis_up`	service	redis_up == 0 for 1 min
`inventory_reservations_active`, `inventory_reservations_expired_total`	warehouse	expired rate spike (> 3× baseline)
`inventory_invariant_drift`	product, warehouse	> 0
`session_events_total`, `session_expiry_reconciled_total`	type, detected_by	reconciled share > 10% (notifications being missed)
`auth_refresh_reuse_total`, `auth_login_failures_total`	—	reuse > 0 (security page); failures > 100/min
`pg_pool_waiting`, `db_query_duration_seconds`	service	pool waiting > 0 for 2 min

16.4 Behaviour under failure

Failure	Expected behaviour	Recovery
PostgreSQL unavailable	Affected services fail readiness; gateway returns 503 `SERVICE_UNAVAILABLE` for their routes; workers pause and retry with backoff; no data written to Redis as truth	Automatic on reconnect; sagas resume from `saga_state`
Redis unavailable	Catalog reads hit PG; sessions validated via PG (slower); rate limit per-instance fallback; keyspace notifications missed → reconciler covers; job locks unavailable → jobs still safe (REDIS-025)	Automatic; reconciler emits missed expiries
RabbitMQ unavailable	User-facing writes succeed; events accumulate in outbox; notifications/audit delayed; consumers reconnect	Relay drains backlog; no loss
gRPC `DEADLINE_EXCEEDED`	Queries: retry within budget then 504; commands: outcome unknown → idempotent retry / status query / reconciler (Section 7.5)	Reconciler resolves within 15 min or escalates
Duplicate events	Consumers ack without side effects via `processed_events`	n/a

Failure	Expected behaviour	Recovery
Partial order failure	Saga persists step; compensations idempotent; reservation TTL guarantees stock release even if Order is down	Reconciler + TTL sweeper
Worker restart mid-processing	Unacked message redelivered; DB transaction either committed (dedup on redelivery) or rolled back (reprocess)	Automatic
Poison message	Classified permanent → DLQ after 1 attempt, or after 10 transient attempts	Fix + replay CLI

17. UI/UX specifications

The dashboard is a thin, role-aware client over the gateway: it hides what a role cannot do, but every rule is still enforced by the backend, and every screen shows the system's real asynchronous state (pending sagas, queued events) rather than pretending writes are instant.

17.1 Frontend requirements

ID	Requirement	Priority
UX-001	SPA with React 19 + TypeScript + Vite, TanStack Query (server state), React Router, Radix UI primitives + Tailwind tokens; served by an nginx static container behind the gateway origin.	Must
UX-002	Navigation and actions derived from the `perms` claim; hidden actions are still rejected server-side (verified by TEST-038 and AC-041).	Must
UX-003	Access token in memory only; silent refresh via HttpOnly cookie 60 s before expiry; on 401 after refresh → redirect to login with "session ended" message.	Must
UX-004	Every list has loading skeleton, empty state with next action, error state showing the correlation ID and retry.	Must
UX-005	Forms validated client-side with the same Zod schemas (shared package) and display server field errors from the `errors[]` array.	Must
UX-006	Mutations send an `Idempotency-Key` generated per form submission (reused on retry).	Must
UX-007	WCAG 2.2 AA: keyboard navigation, visible focus, 4.5:1 contrast, labelled inputs, `aria-live` for toasts and status changes.	Must
UX-008	Responsive: ≥ 1280 px full layout; 768–1279 px collapsible sidebar; < 768 px read-only lists and order detail (adjustments desktop-only).	Should

ID	Requirement	Priority
UX-009	Order status updates by polling `/orders/{id}/status` every 2 s for up to 30 s, then every 15 s; Server-Sent Events are Could.	Must
UX-010	Light and dark themes from the token set below.	Could

17.2 Design tokens

Token	Light	Dark	Use
`color.bg`	#F7F8FA	#0F1115	App background
`color.surface`	#FFFFFF	#171A21	Cards, tables
`color.text`	#111827	#E5E7EB	Body text
`color.muted`	#6B7280	#9CA3AF	Secondary text
`color.primary`	#2563EB	#60A5FA	Primary actions, links
`color.success`	#15803D	#4ADE80	CONFIRMED, healthy
`color.warning`	#B45309	#FBBF24	Low stock, PAYMENT_UNKNOWN, pending
`color.danger`	#B91C1C	#F87171	FAILED, revoked, DLQ
`font.family`	Inter, system-ui	same	All UI; JetBrains Mono for IDs/SKUs
`font.size`	12 / 14 / 16 / 20 / 24	same	caption / body / emphasis / h2 / h1
`space`	4-pt scale: 4, 8, 12, 16, 24, 32	same	Padding, gaps
`radius`	6 (inputs), 8 (cards)	same	
`elevation`	1 px border `#E5E7EB`, no heavy shadows	border `#2A2F3A`	

Core components: AppShell (sidebar + top bar with notification bell and session menu), DataTable (server cursor pagination, column sort, filter bar, URL-synced state), StatusBadge (one colour per order/payment/reservation status), MoneyText (minor units → locale format), EntityDrawer (detail side panel), ConfirmDialog (destructive actions require typing reason), Timeline (status history), KPI tile, Toast.

17.3 Screens

Screen	Primary role(s)	Data shown	Actions (permission)	Filters / pagination	States & validation
Login	all	email, password	sign in	—	generic "invalid credentials"; lockout message with retry time; 429 banner
My sessions	all	device, IP prefix, created, last seen, current flag	revoke one / all others (`identity:session` D(O))	—	revoke confirm; current session marked
Dashboard	role-specific	customer: recent orders; inventory: low-stock count, reservations active, movements today; order mgr: orders by status, REQUIRES_ATTENTION list; admin: service health tiles, DLQ/outbox counters	quick links	date range (today / 7 d / 30 d)	tiles show "data delayed" if audit lag > 5 min
Products	catalog_mgr (W), others (R)	SKU, name, category, price, status, updated	create, edit, archive, upload media (`catalog:product`)	text, category, status, price range; sort; cursor	SKU format, price > 0; 409 SKU conflict shown inline
Categories	catalog_mgr	tree (depth ≤ 4), product counts	add child, rename, archive	—	cannot archive with active products

Screen	Primary role(s)	Data shown	Actions (permission)	Filters / pagination	States & validation
Warehouses	inventory_mgr	code, name, priority, status, SKUs stocked	create, edit, deactivate (`inventory:warehouse`)	status	code unique
Inventory	inventory_mgr, wh_operator (assigned only)	per SKU × warehouse: on hand, reserved, available, threshold, low badge	adjust, transfer	warehouse, category, low-stock only, SKU search	available never shown negative; adjustment preview shows resulting balance
Stock adjustment / transfer (drawer)	inventory_mgr, wh_operator	current balances	submit (`inventory:adjust` / `transfer`)	—	delta ≠ 0; reason required; note ≥ 3 chars; 409 insufficient shows live available
Movement history	inventory_mgr, auditor	time, type, delta, actor, reference, correlation ID	export CSV (Could)	warehouse, SKU, type, date	append-only, no edit controls
Orders	order_mgr, customer (own)	id, customer, total, status, created, updated	open detail	status, date range, total range, customer	status badge incl. PAYMENT_UNKNOWN / REQUIRES_ATTENTION
Order detail	order_mgr, customer	lines with price snapshot, totals, reservation & payment ids, status timeline, notifications sent	cancel (`order:cancel`), refund (`payment:refund`), fulfil, retry saga / force fail (admin)	—	cancel disabled with reason when state disallows; live status polling

Screen	Primary role(s)	Data shown	Actions (permission)	Filters / pagination	States & validation
Payments	finance, order_mgr	payment id, order, amount, status, decline code, refunded	partial / full refund	status, date, amount	refund ≤ remaining; reason required
Payment simulator	developer/admin (non-prod)	scenario tokens and their effect	create test order with scenario	—	hidden when `NODE_ENV=production`
Notifications	all	title, body, time, read state	mark read, mark all read	unread only	unread count in bell
Audit log	auditor, admin	time, type, actor, resource, correlation ID, JSON data	open trace link (Jaeger)	type, actor, resource, date	read-only; JSON viewer redacts nothing because events contain no secrets
Session analytics	auditor, admin	daily created / revoked / expired (by detectedBy), reuse incidents	—	date range	chart + table
System health	admin, auditor	per-service readiness, dependency status, queue depths, DLQ, outbox backlog, cache hit rate	link to Grafana / RabbitMQ UI	—	auto-refresh 15 s; stale indicator

17.4 Wireframe descriptions

- **AppShell:** left sidebar (240 px) with sections Overview, Catalog, Inventory, Orders, Payments, Notifications, Audit, Admin — each shown only if the user has a read permission in it; top bar with global search (SKU / order id), notification bell with unread count, avatar menu (My sessions, Sign out).
- **Order detail:** header row (Order #, StatusBadge, total, created); left column (8/12) line-item table then status Timeline; right column (4/12) cards: Payment (status, method token, refunds), Reservation (id, expires-at countdown while `RESERVED`), Actions (Cancel, Refund, Fulfil) with disabled-reason tooltips.

- **Inventory:** filter bar on top; DataTable with sticky first column (SKU + name); available column right-aligned with warning colour when ≤ threshold; row action "Adjust" opens EntityDrawer with a before/after balance preview.
- **System health:** grid of service tiles (green/amber/red with dependency list), below a row of KPI tiles (queue ready, DLQ, outbox oldest age, cache hit %), and a REQUIRES_ATTENTION order table.

17.5 Seed / sample data

Entity	Sample
Warehouses	`BLR-01` Bengaluru (priority 10), `DEL-01` Delhi (20), `MUM-01` Mumbai (30)
Categories	Electronics › Accessories › Cables; Office › Furniture; Office › Stationery
Products	`ACC-USBC-1M` USB-C cable 1 m, ₹499.00 (49900), threshold 25; `OFF-CHAIR-ERG` Ergonomic chair, ₹12,999.00, threshold 5; `STA-A4-500` A4 paper 500 sheets, ₹349.00, threshold 50
Balances	USB-C cable: BLR 120 / DEL 40 / MUM 0; chair: BLR 6 / DEL 2
Users	`customer1@demo.local`, `ops@demo.local` (order_manager), `stock@demo.local` (inventory_manager), `floor.blr@demo.local` (warehouse_operator, BLR-01), `finance@demo.local`, `audit@demo.local`, `admin@demo.local` — passwords generated by `make seed` and printed once, never committed

18. Non-functional requirements

Targets are sized for the MVP load in assumption A-5 (50 RPS sustained, 200 RPS burst) on the reference Compose environment, measured by the nightly k6 suite.

ID	Category	Requirement	Priority
NFR-001	Latency	Gateway p95: reads ≤ 150 ms (cached product ≤ 50 ms), `POST /orders` (202) ≤ 300 ms; p99 ≤ 2× p95.	Must
NFR-002	Saga latency	p95 order `PENDING → CONFIRMED` ≤ 3 s with `pm_ok`.	Must
NFR-003	Throughput	200 RPS order creation burst for 60 s with < 0.5% 5xx and zero invariant violations.	Must
NFR-004	Correctness	0 oversell, 0 double charges, 0 negative balances under all E2E and chaos scenarios.	Must
NFR-005	Event latency	p95 outbox commit → consumer processed ≤ 2 s with broker healthy.	Should

ID	Category	Requirement	Priority
NFR-006	Durability	No committed business event lost across broker restart, relay crash or consumer crash (at-least-once).	Must
NFR-007	Availability (target, not SLA)	Each service tolerates restart of any single dependency without manual intervention; recovery ≤ 60 s after dependency returns.	Must
NFR-008	Scalability	Every service and worker horizontally scalable to ≥ 3 replicas without code change (SKIP LOCKED, idempotent consumers, stateless handlers).	Must
NFR-009	Startup	Each service ready ≤ 20 s after its dependencies are healthy; full `make up` ≤ 3 min (cached images).	Should
NFR-010	Security	Revocation effective ≤ 1 s; no secrets in repo/images; 0 ERROR Semgrep; 0 critical CVEs with fix.	Must
NFR-011	Maintainability	Strict TS, no `any` in `domain/` (lint), cyclomatic complexity ≤ 15 per function, ADR for every significant decision.	Should
NFR-012	Observability	100% of requests carry correlation + trace IDs; every alert has a runbook.	Must
NFR-013	Portability	Runs on Linux, macOS (Apple Silicon) and Windows WSL2; images multi-arch (amd64, arm64).	Should
NFR-014	Accessibility	Dashboard meets WCAG 2.2 AA on the screens in 17.3 (axe-core automated check, 0 serious violations).	Should
NFR-015	Data retention	Retention policy DB-011 enforced by scheduled jobs with metrics.	Should
NFR-016	Documentation	README per service (purpose, config, RPCs, events, run/test), root quick-start, OpenAPI, AsyncAPI, Postman, runbooks.	Must

19. Acceptance criteria

46 acceptance criteria cover every Must requirement; each is automated and tagged in test names (TEST-042).

ID	Given / When / Then	Traces to	Verified by
AC-001	**Given** a new email **when** a customer registers and logs in **then** the response has a 10-min ES256 access token, a refresh cookie with HttpOnly/Secure/SameSite=Strict, and a session row `ACTIVE` in PG and Redis.	FR-001, FR-002	integration (identity)
AC-002	**Given** a valid refresh token **when** it is used **then** a new token of generation n+1 is returned and the old one is marked `used_at`.	FR-003, REDIS-015	unit + integration
AC-003	**Given** a refresh token already rotated more than 10 s ago **when** it is presented again **then** 401 is returned, every token in the family and the session are revoked, and `identity.session.revoked{reason: token_reuse}` plus an audit event are produced.	FR-004, REDIS-016	TEST-033
AC-004	**Given** a user with sessions on devices A and B **when** A revokes B **then** B's next protected request returns 401 within 1 s.	FR-005, FR-006, REDIS-013	TEST-032
AC-005	**Given** a user with 3 sessions **when** `GET /me/sessions` **then** exactly 3 active sessions are listed with the current one flagged and no token material exposed.	FR-005	integration
AC-006	**Given** a session whose idle TTL elapses while the Identity subscriber is disconnected **when** the reconciler runs **then** exactly one `identity.session.expired{detectedBy: reconciler}` event and one audit row exist within 6 min.	FR-007, FR-052, REDIS-022	TEST-018, TEST-034
AC-007	**Given** a product with SKU `ACC-USBC-1M` **when** another product is created with `acc-usbc-1m` **then** 409 `SKU_CONFLICT`; **and when** a category with active products is deleted **then** 409.	FR-011, FR-012	integration
AC-008	**Given** product P cached **when** its price is updated **then** the next `GET /products/P` returns the new price (cache key deleted) and `cache_requests_total{result=miss}` increments once.	FR-014, REDIS-001/002	integration

ID	Given / When / Then	Traces to	Verified by
AC-009	**Given** 250 products **when** listing with `limit=100` **then** three pages via `nextCursor` cover all rows exactly once, stable under concurrent inserts.	FR-013	integration
AC-010	**Given** on_hand 50 for a SKU **when** 200 concurrent single-unit orders are placed **then** exactly 50 reach `CONFIRMED`, `available` is never negative at any sampled point, and ledger sum equals on_hand.	FR-018, FR-019, DB-013/014, G-1	TEST-016, TEST-027
AC-011	**Given** a reservation with TTL 60 s (test config) and no payment **when** the TTL passes **then** within 30 s it is `EXPIRED`, `reserved` is decremented, `inventory.stock.released{EXPIRED_TTL}` is published and the order ends `FAILED(RESERVATION_EXPIRED)`.	FR-021, FR-022, SAGA-010	integration + E2E
AC-012	**Given** a warehouse operator assigned to BLR-01 **when** they adjust stock by −5 with reason DAMAGE **then** one DAMAGE movement with actor and correlation ID is written; adjusting DEL-01 returns 404; an adjustment making available negative returns 409.	FR-017, FR-023, FR-025	integration
AC-013	**Given** 10 units in BLR-01 **when** 4 are transferred to DEL-01 **then** both balances change in one transaction with paired movements sharing `reference_id`; a failure mid-way leaves both unchanged.	FR-024, DB-018	integration
AC-014	**Given** threshold 25 and available 30 **when** available drops to 24 and then 20 **then** exactly one `inventory.stock.low` is published and one notification per inventory manager is created; after restocking to 31 and dropping to 24 again, a second alert fires.	FR-026, Workflow C	TEST-037
AC-015	**Given** an order created with key K **when** the same payload is sent 5× concurrently with K **then** all return the same order ID and one order exists; **when** a different payload is sent with K **then** 409 `IDEMPOTENCY_KEY_REUSED`.	FR-028, API-008	TEST-026
AC-016	**Given** stock and `pm_ok` **when** an order is placed **then** it is `CONFIRMED` within 5 s, on_hand reduced, one `SUCCEEDED` payment, events `order.created`,	FR-020, FR-031, FR-036	TEST-025

ID	Given / When / Then	Traces to	Verified by
	`order.confirmed` delivered, and one customer notification.		
AC-017	**Given** `pm_declined` **when** an order is placed **then** it ends `FAILED`, the reservation is `RELEASED`, available equals the pre-order value, and the customer has a failure notification.	FR-031, Workflow B	TEST-028
AC-018	**Given** `pm_slow_success` (captures, then exceeds the 3 s deadline) **when** an order is placed **then** the order ends `CONFIRMED` with exactly one `SUCCEEDED` payment row and no refund.	GRPC-013/014, FR-037, FR-039	TEST-029
AC-019	**Given** a `CONFIRMED` order **when** the customer cancels **then** a full refund succeeds, `RETURN` movements restore stock, order is `CANCELLED`, and a repeat cancel with the same key returns 200 with the same result.	FR-032, SAGA-007	TEST-031
AC-020	**Given** a `FAILED` order **when** cancel or fulfil is requested **then** 409 `INVALID_STATE_TRANSITION` and no history row is added; every legal transition adds exactly one history row.	FR-030, FR-033	unit TEST-002 + integration
AC-021	**Given** a captured ₹1,000 payment **when** refunds of ₹600 and ₹600 are requested **then** the first succeeds and the second is rejected; a retried ₹600 with the same refund key returns the original result.	FR-038	integration
AC-022	**Given** `order.confirmed` delivered 3 times **when** Notification consumes **then** exactly one notification exists and the unread count increments by one.	FR-041–043, MQ-006	TEST-019, TEST-036
AC-023	**Given** a consumer failing transiently on every attempt **when** a message arrives **then** after 10 attempts it is in the DLQ with `x-death` and `x-last-error`, the DLQ alert fires, and the replay CLI returns it to processing.	MQ-007–009, FR-051	TEST-020
AC-024	**Given** RabbitMQ paused **when** 100 orders are placed **then** order creation returns 202 for all, `outbox_backlog` grows, and after unpausing all events are delivered within 60 s.	MQ-002, MQ-011, G-3	TEST-021, TEST-035

ID	Given / When / Then	Traces to	Verified by
AC-025	**Given** the relay crashes after broker ack but before marking published **when** it restarts **then** the event is republished with the same `eventId` and consumers do not duplicate side effects.	MQ-003, MQ-006	integration (fault injection)
AC-026	**Given** Inventory delayed by 3 s via toxiproxy **when** Order calls `ReserveStock` **then** the call ends `DEADLINE_EXCEEDED` at 2 s, the gateway path shows 504 only for synchronous queries, and the saga retries with the same key.	GRPC-002, GRPC-007	TEST-022 + E2E
AC-027	**Given** a gRPC client without a valid client certificate (or not on the RPC allow-list) **when** it calls `Payment.ProcessRefund` **then** the call fails `UNAUTHENTICATED`/`PERMISSION_DENIED`.	GRPC-005, SEC-012	TEST-022
AC-028	**Given** any 4xx/5xx **then** the body matches the Problem schema with a correlation ID, and contains no stack trace, SQL or internal hostname (checked by regex over the E2E corpus).	API-004, SEC-015	TEST-024
AC-029	**Given** 11 login attempts from one IP within a minute **then** the 11th returns 429 with `Retry-After`; **and** 5 failed logins on one account trigger progressive delay.	FR-008, API-010	TEST-024
AC-030	**Given** each role in the permission matrix **when** calling every endpoint **then** responses match the matrix (allowed → 2xx/4xx business; denied → 403/404), including cross-customer order access returning 404.	FR-AUTHZ-01–03, SEC-002/003, FR-009	permission matrix E2E + TEST-038
AC-031	**Given** Redis stopped **when** products are read **then** responses are 200 from PostgreSQL, Catalog `/readyz` stays 200, and `cache_errors_total` increases.	REDIS-006	TEST-039
AC-032	**Given** Redis stopped **when** a user with a valid session calls a protected route **then** it succeeds via PG fallback; **given** PG and Redis both down **then** 503, never 200.	REDIS-012	integration
AC-033	**Given** PostgreSQL stopped **then** affected services report `/readyz` 503 but `/livez` 200, are not	OBS-001/002, NFR-007	E2E chaos

ID	Given / When / Then	Traces to	Verified by
	restarted by Docker, and serve again within 60 s of PG returning.		
AC-034	**Given** a fresh clone with Docker **when** `make bootstrap && make up` **then** all containers are healthy within 3 min, run as non-root (`docker inspect` UID ≠ 0), and only the documented ports bind to 127.0.0.1.	DOCKER-001–008, SEC-019/020	CI compose smoke test
AC-035	**Given** a PR introducing `pool.query(…${id})` **then** Semgrep reports `sf-sql-string-concat` as ERROR and the merge is blocked; a `nosemgrep` without justification also fails.	SAST-001–007	CI fixture PR
AC-036	**Given** coverage below the TEST-040 thresholds in a security-critical module **then** CI fails with the module named.	TEST-040/041	CI
AC-037	**Given** RabbitMQ down **then** Order `/readyz` stays 200 (soft dep) while Notification `/readyz` is 503; `/ops/health` shows both accurately.	OBS-002, FR-048, FR-050	E2E
AC-038	**Given** a request with `X-Correlation-Id: C` **when** it creates an order **then** C appears in gateway, order, inventory, payment logs, in the outbox headers, in the notification consumer log, and the trace spans one trace ID.	FR-049, OBS-004/005	E2E log assertion
AC-039	**Given** a role change, stock adjustment, refund, login failure and refresh reuse **then** each produces exactly one `audit_events` row, and the app DB role cannot UPDATE or DELETE audit rows.	FR-045, FR-046, SEC-017	integration
AC-040	**Given** a presigned upload **when** a client uploads an HTML file renamed `.png` **then** finalize marks media `REJECTED`; direct unauthenticated GET of the object returns 403.	FR-015, SEC-013	TEST-023
AC-041	**Given** a customer token **when** the UI is bypassed and `POST /inventory/adjustments` is called directly **then** 403.	UX-002, SEC-002	E2E
AC-042	**Given** built images and repo **then** Gitleaks and Trivy secret scan find 0 secrets, and `ProcessPayment` rejects a 16-digit card-like string with `INVALID_ARGUMENT`.	SEC-011, DOCKER-012, FR-040	CI + unit

ID	Given / When / Then	Traces to	Verified by
AC-043	**Given** an order saga mid-payment **when** the order worker receives SIGTERM **then** it finishes or safely persists the current step within 20 s, exits 0, and another replica completes the saga.	DOCKER-006, OBS-010	E2E chaos
AC-044	**Given** a day's events **when** the daily summary job runs twice **then** counters are identical (idempotent via `summary_contributions`).	FR-047	integration
AC-045	**Given** `limit=500` **then** 400 `LIMIT_TOO_LARGE`; unknown filter param → 400.	API-006, API-007, FR-034	TEST-024
AC-046	**Given** an order placed at ₹499 **when** the product price changes to ₹599 **then** the order lines still show ₹499 and totals are unchanged.	FR-016, FR-029	integration

20. Risks and mitigations

The biggest risks are scope (eight services for a learning team) and subtle distributed bugs that only show under concurrency; both are mitigated by phasing and by making failure scenarios part of the definition of done.

ID	Risk	Likelihood	Impact	Mitigation
R-01	Scope too large for the team/timeline (8 services + workers + UI)	High	High	Strict phase gates (Section 21); Audit and Notification can start as one "Engagement" service if needed (decision OQ-03); UI kept to 17.3 Must screens
R-02	Deadlocks or lock contention on hot SKUs under burst	Medium	High	Deterministic lock order, short transactions, lock_timeout 2 s, retry on 40P01; k6 hot-SKU profile in nightly
R-03	Saga stuck states not covered by reconciler	Medium	High	Exhaustive state-machine tests; reconciler SLA per state; `REQUIRES_ATTENTION` alert; chaos E2E
R-04	Duplicate side effects from at-least-once delivery	Medium	High	`processed_events` in same transaction; unique constraints on natural keys (`UNIQUE(source_event_id, recipient_id)`); dedup tests
R-05	Outbox table growth / relay lag	Medium	Medium	Partial index, 7-day purge, backlog alert, scalable relays with SKIP LOCKED

ID	Risk	Likelihood	Impact	Mitigation
R-06	Missed Redis expiry notifications mistaken for reliable	High	Medium	Reconciler is the guarantee; metric of reconciled share; documented in REDIS-019–022
R-07	gRPC/proto breaking changes between independently deployed services	Medium	High	Buf breaking check; additive-only rule; `v2` package for breaks; consumer contract tests
R-08	mTLS/cert setup friction blocks local development	Medium	Medium	`make certs` generates dev CA automatically; documented troubleshooting; insecure mode only in unit tests
R-09	Semgrep noise leads to blanket suppressions	Medium	Medium	Justification-required rule; quarterly rule tuning; diff-aware scans
R-10	Flaky integration/E2E tests erode trust in CI	High	Medium	Injected clock, `eventually()` polling, Testcontainers reuse per suite, quarantine with ticket and 7-day fix SLA
R-11	Laptop resource limits (8 GB) with full stack + observability	Medium	Medium	Resource caps (DOCKER-009); observability as optional profile; single Postgres instance
R-12	Security regressions in auth code	Low	High	95% coverage gate on auth modules; CODEOWNERS security review; JWT negative test suite
R-13	Money arithmetic errors	Low	High	Integer minor units only; `bigint`; DB check constraints; property tests on totals
R-14	Library version drift / EOL during project	Medium	Low	Version ADR; Renovate; Node LTS schedule tracked

21. Implementation phases and deliverables

Five gated phases take the project from skeleton to production-ready; each phase ends only when its exit gate's acceptance criteria pass in CI. Durations are indicative for a team of 3–4 backend engineers plus one part-time frontend engineer (about 12 weeks) and must be confirmed (OQ-02).

```
flowchart LR
  P1["Phase 1 - Foundation<br/>~2 wks"] --> G1{{"Gate 1<br/>make up green, CI
skeleton,<br/>migrations, AC-034"}}
  G1 --> P2["Phase 2 - Core services<br/>~3 wks"] --> G2{{"Gate 2<br/>AC-007..013,
AC-020, AC-046,<br/>AC-010 concurrency"}}
  G2 --> P3["Phase 3 - Distributed workflows<br/>~3 wks"] --> G3{{"Gate
3<br/>AC-014..019, AC-021..026,<br/>AC-043"}}
  G3 --> P4["Phase 4 - Security and quality<br/>~2 wks"] --> G4{{"Gate
4<br/>AC-001..006, AC-027..032,<br/>AC-035, AC-036, AC-039..042"}}
  G4 --> P5["Phase 5 - Production readiness<br/>~2 wks"] --> G5{{"Gate 5 =
Release<br/>all AC, NFR-001..004,<br/>Definition of Done"}}
```

Phase	Scope	Deliverables
1. Foundation	Monorepo (pnpm + Turborepo), strict TS configs, `packages/platform` (config, logger, errors, health), proto toolchain (Buf + ts-proto), node-pg-migrate per service, Postgres init (7 DBs + roles), Compose with infra + empty services, ESLint/Prettier, Gitleaks, Semgrep baseline, CI skeleton, ADRs 0001–0005	Repo skeleton; `make up` with healthy empty services; CI running lint/type/unit/semgrep; version ADR
2. Core services	Catalog, Inventory, Order (create + read, synchronous happy path), Payment simulator; gateway routing + error mapping; gRPC with deadlines, mTLS (dev CA), status mapping; inventory transactions, reservation SQL, movement ledger; cursor pagination; OpenAPI generation	REST + gRPC contracts v1; concurrency integration test (TEST-016); Postman collection v1
3. Distributed workflows	RabbitMQ topology, `mq-kit`, outbox + relay, idempotent consumers, retry tiers + DLQ, replay CLI; saga worker + state machine + compensations + reconciler; reservation sweeper; Notification + Audit services; low-stock workflow	AsyncAPI doc; E2E A/B/C; chaos tests for broker/worker failures
4. Security and quality	Identity (register/login, ES256, rotation, reuse detection, sessions, revocation, keyspace + reconciler), RBAC + permission matrix tests, rate limiting, helmet/CORS/CSRF, Redis cache-aside + stampede control, private media uploads, custom Semgrep rules + tests, coverage gates, Trivy	Security test suite; Semgrep custom rules; coverage thresholds enforced; threat model
5. Production readiness	OTel tracing, Prometheus metrics, Grafana dashboards, alert rules, runbooks; graceful shutdown; container hardening (read-only, cap_drop, distroless); k6 load profiles; failure-injection suite; dashboard UI (17.3 Must screens); docs	Release candidate images signed with SBOM; dashboards; runbooks; UI; final PRD traceability report

Recommended working sequence: approve this PRD → produce the service-by-service technical design (one doc per service: data model, RPCs, events, failure modes) → generate the backlog with one ticket per requirement ID, each ticket citing its AC → start Phase 1. Reviewing gRPC vs RabbitMQ vs Redis vs PostgreSQL boundaries at the design stage is cheaper than refactoring them in Phase 3.

22. Definition of Done

A requirement, service or release is done only when every applicable item below is true and verifiable in CI or the repository.

Per requirement / ticket

- ☐ Code merged to `main` through a reviewed PR (CODEOWNER approval for auth, payment, inventory, saga code).
- ☐ Linked acceptance criterion automated and tagged `[AC-xxx]`; failure paths tested, not only the happy path.
- ☐ Unit + integration tests pass; coverage thresholds (TEST-040) hold.
- ☐ Semgrep 0 ERROR, Gitleaks 0, ESLint 0 errors, `tsc` 0 errors, Buf lint/breaking clean.
- ☐ OpenAPI / proto / event schema updated and regenerated artifacts committed; Postman collection regenerated.
- ☐ Logs, metrics and trace spans added for new flows; correlation ID verified.
- ☐ Idempotency, timeouts and retry behaviour stated for any new mutation or remote call.
- ☐ README / ADR / runbook updated where behaviour or operations changed.

Per service

- ☐ Own Dockerfile (multi-stage, non-root, healthcheck), own migrations, own DB role, Zod config schema, `.env.example` entry.
- ☐ `/livez`, `/readyz`, gRPC health, `/metrics`, graceful shutdown verified (AC-043).
- ☐ No cross-database access (verified by DB role grants) and no undeclared gRPC dependencies (ARCH-006 lint).

Per release

- ☐ All Must requirements implemented; all 46 ACs green in the merge queue and nightly chaos suite.
- ☐ NFR-001–004 met by the k6 run attached to the release.
- ☐ Trivy: 0 CRITICAL with fix; HIGH triaged. SBOM and image signatures published.
- ☐ Grafana dashboards and alert rules load from the repo; every alert links a runbook.
- ☐ Fresh-clone `make bootstrap && make up && make seed && make e2e` succeeds on Linux and macOS.
- ☐ Open questions in Section 23 either answered or explicitly deferred with owner and date.

23. Open questions for stakeholders

These decisions change scope or design and need an owner's answer before or during Phase 1; the PRD's current default is stated for each.

ID	Question	Current default in this PRD	Needed by
OQ-01	Monorepo or multi-repo?	Monorepo (pnpm + Turborepo) with independent images and versions	Phase 1 start
OQ-02	Team size and calendar for the 5 phases?	3–4 backend + 1 part-time frontend, ~12 weeks (indicative)	Phase 1 start
OQ-03	Keep Audit & Reporting as a separate service, or merge with Notification for MVP?	Separate audit-svc	Phase 3
OQ-04	Currency and locale: single INR deployment, or multi-currency?	Single currency, INR default, integer minor units	Phase 2
OQ-05	Reservation TTL and payment SLA (15 min / 3 s deadline) acceptable?	15 min TTL, 3 s payment deadline	Phase 2
OQ-06	Can customers cancel after `CONFIRMED`, and is fulfilment in scope beyond a manual flag?	Cancel allowed until `FULFILLED`; fulfilment = manual flag	Phase 2
OQ-07	Which jurisdictions and data-protection obligations apply (e.g. India DPDP Act, GDPR)? Retention periods?	No compliance claimed; retention per DB-011	Phase 4
OQ-08	Refund limit for `order_manager` before `finance` approval is required?	Configurable, default ₹10,000	Phase 3
OQ-09	Should order creation stay asynchronous (202 + polling) or block until confirmed for API clients?	202 + polling/notification; gateway may return final state if done in 2 s	Phase 2
OQ-10	Production target platform (Kubernetes, ECS, VMs) and secret manager?	Out of MVP; Compose + Docker secrets locally	Phase 5
OQ-11	Is MFA for staff required for MVP?	Could (FR-010)	Phase 4
OQ-12	Jest or Vitest as the single test runner?	Vitest	Phase 1 start